Automated Glaucoma Risk Assessment

Deep Learning-Based Optic Disc and Cup Segmentation with CDR

Final Project Report: University of London

Course: DSM500-2025-Oct

Date: 30 March 2026

Rebecca Ismay Tyler

Abstract

Glaucoma is a leading cause of irreversible blindness worldwide, with early detection relying critically on accurate measurement of the cup-to-disc ratio, a quantitative biomarker derived from the morphology of the optic nerve head in retinal fundus images. Manual CDR estimation is time-consuming and subject to significant inter-grader variability, motivating the development of automated segmentation pipelines capable of producing reproducible, clinically relevant measurements at scale.

This study presents an automated end-to-end pipeline for optic disc and cup segmentation, CDR estimation, and glaucoma risk classification. Three deep learning architectures are evaluated: a lightweight MobileNetV2-UNet, a ResUNet++ Lite incorporating LayerNorm and attention gates, and a full ResUNet++ with BatchNorm, squeeze-and-excitation blocks, and focal Tversky loss optimisation. All models were trained and evaluated across six publicly available retinal fundus datasets; CRFO, DRISHTI-GS, G1020, ORIGA, PAPILA, and REFUGE, which 2084 images spanning population screening and clinical referral populations. Segmentation performance was assessed using Dice coefficient and Intersection over Union for both optic disc and optic cup. CDR values derived from predicted segmentation masks were subsequently evaluated against expert-annotated ground truth using Pearson correlation and used as predictors in a logistic regression model for glaucoma risk classification on the DRISHTI-GS dataset.

ResUNet++ achieved the highest segmentation performance, attaining a macro-Dice of 0.81 with the optic cup Dice at 0.84 and a validation loss of 0.134. MobileNetV2-UNet demonstrated competitive performance at significantly reduced computational cost, achieving a validation optic disc Dice of 0.78 and optic cup Dice of 0.86. The ResUNet++ Lite model was competitive with validation cup Dice being the highest at 0.86, with disc Dice stabilizing at 0.78. These results demonstrate that deep learning-derived CDR values can reliably approximate expert assessments, supporting the feasibility of automated glaucoma screening pipelines in both resource-constrained and specialist clinical settings.

Contents

List of Figures 3
List of Acronyms 3
1. Project Overview 5
2. Research Hypotheses 6
2.1 Primary Hypothesis 6
2.2 Secondary Hypotheses 6
3. Literature Review 6
3.1 Foundations of Encoder–Decoder Segmentation 6
3.6 CDR Estimation and Clinical Context 7
3.7 Datasets and Evaluation Benchmarks 7
3.8 Segmentation Studies 8
4. Positioning of the Current Work 8
5. Methodology 8
5.2 Dataset 9
5.2 Data Preprocessing Pipeline 10
5.4 Evaluation Metrics 10
5.5 CDR Computation 11
5.6 Dataset Bias and Limitations 12
5.7 Computational Resources 12
6.2.1 Training Configuration 14
6.3.1 Training Configuration 14
6.4 Loss Functions 14
6.8 Model Comparison 15
6.8.1 MobileNetV2-UNet 15
6.8.2 ResUNet++ Lite 15
6.8.3 ResUNet++ Medium 16
7. Evaluation 16
7.1 Cross-Dataset Performance Patterns 17
7.2 Scatter Plots: Predicted vs. Ground Truth CDR 17
7.3 Dice Distribution Plots 18
7.4 Bland-Altman Analysis 18
7.5 Box Plots 18
7.6 CDR Regression on DRISHTI-GS 18
7.4 Regression with DRISHTI 19
8. Model Summary and Comparison 20
10.1 Longitudinal Dataset Integration 24
10.3 Model Ensembling 25

10.4 Expanded Dataset Diversity 25
10.5 Architectural Refinement for Cup Segmentation 25
11. Conclusion 25
Bibliography 27
Appendix 1: Code 28
Data Processing Pipeline 28
Models 33

List of Figures

Figure 1: Input image with raw OD/O C mask and overlay with contours
Figure 2: Input image with predicted OD/OC mask and overlay with contours and predicted CDR
Figure 3: Summary of Dataset pipeline
Figure 4: Scatter plots for three models showing disc Dice and cup Dice.
Figure 5: Disc and cup distribution curves.
Figure 6: Bland-Altman Scatter plot comparing all three models
Figure 7: Box Plots comparing all three models
Figure 8: CDR Regression on DRISHTI dataset.
Figure 9: TensorFlow REFUGE dataset creation and splits

List of Tables

Table 1Summary of metrics and equations used in this study
Table 2: Dataset Summary
Table 3: Summary of similarities and differences between models created for this study
Table 4: Comparison of the three model architectures.
Table 5: Summary of models
Table 6: Comparison table of disc and cup stability, CDR accuracy and reliability.

List of Acronyms

ACRONYM	DESCRIPTION
ACRIMA	ACRIMA Glaucoma Dataset
ADAM	Adaptive Moment Estimation
ASPP	Atrous Spatial Pyramid Pooling
AUC	Area Under the Curve
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BN	Batch Normalisation
CDR	Cup-to-Disc Ratio
CLAHE	Contrast Limited Adaptive Histogram Equalization

CNN	Convolutional Neural Network
CRFO	Colour Retinal Fundus image for Optic
DDSC-Net	Densely Connected Depthwise
DDSC-Net	Densely Connected Depthwise Separable Convolution Network
DL	Deep Learning
DRIONS-DB	Digital Retinal Images for Optic Nerve Segmentation Database
DRISHTI-GS	DRISHTI Glaucoma Screening Dataset
DSC	Dice Similarity Coefficient
EMA	Exponential Moving Average
FLOPs	Floating Point Operations Per Second
FN	False Negative
FP	False Positive
fp16	16-bit Floating Point
fp32	32-bit Floating Point
G1020	Glaucoma 1020 Dataset
GL-Net	Global-Local Network
GPU	Graphics Processing Unit
IDRiD	Indian Diabetic Retinopathy Image Dataset
INT8	8-bit Integer
IoU	Intersection over Union
IOP	Intraocular Pressure
LN	Layer Normalisation
LR	Learning Rate
M-Net	Multi-scale
U-Net	Variant
MCAU-Net	Multiresolution Cascaded Attention U-Net
MICCAI	Medical Image Computing and Computer-Assisted Intervention
ML	Machine Learning
MRI	Magnetic Resonance Imaging
NPZ	NumPy Compressed File Format
OC	Optic Cup
OCT	Optical Coherence Tomography
OD	Optic Disc
ORIGA	Online Retinal Fundus Image Database for Glaucoma Analysis
PAPILA	Glaucoma Dataset with Paired Eyes and Clinical Metadata
pOSAL	Patch-based Output Space Adversarial Learning
REFUGE	Retinal Fundus Glaucoma Challenge
ReLU	Rectified Linear Unit
ResNet	Residual Network ResUNet – Residual U-Network

ResUNet++	Residual U-Network Plus Plus
RIGA	Retinal Image Glaucoma Analysis Dataset
RIM-ONE	Retinal Images for Optic Nerve Evaluation
ROC	Receiver Operating Characteristic
ROI	Region of Interest
SE	Squeeze-and-Excitation
SGD	Stochastic Gradient Descent
SMDG-19	Standardized Multi-Channel Dataset for Glaucoma (2019)
SSD	Solid State Drive Stack-U-Net – Stacked U-Net Architecture
T4	Tesla T4 Graphics Processing Unit
TN	True Negative
TP	True Positive
TPU	Tensor Processing Unit
U-Net	U-shaped Convolutional Neural Network
vCDR	Vertical Cup-to-Disc Ratio
VRAM	Video Random Access Memory
XLA	Accelerated Linear Algebra

1. Project Overview

Glaucoma remains one of the leading causes of irreversible blindness worldwide, with early detection critically dependent on accurate interpretation of structural biomarkers derived from retinal fundus imaging. Chief among these is the cup-to-disc ratio, a quantitative measure of optic nerve head morphology whose elevation above clinically established thresholds is strongly associated with glaucomatous damage. Manual CDR estimation is inherently subjective, time-consuming, and subject to significant inter-grader variability, motivating the development of automated, reproducible methods capable of supporting clinical decision-making at scale.

This project addresses that need through an end-to-end pipeline for automated glaucoma risk assessment, integrating deep learning-based optic disc and cup segmentation with statistical modelling for CDR-based classification. Two architectures are evaluated and compared: MobileNetV2-UNet, a lightweight hybrid model combining the efficient feature extraction of MobileNetV2 with the spatial localisation capabilities of UNet, and ResUNet++, a mediumweight architecture incorporating residual connections, attention gates, squeeze-and-excitation blocks, and focal Tversky loss optimisation. Segmentation performance is evaluated across six publicly available retinal fundus datasets, CRFO, DRISHTI-GS, G1020, ORIGA, PAPILA, and REFUGE, spanning population screening and clinical referral populations. CDR values derived from model predictions are subsequently used as inputs to a logistic regression model, trained against expert-graded CDR annotations on the DRISHTI-GS dataset, to assess whether automated segmentation outputs can serve as reliable predictors for glaucoma risk classification.

The dual-stage design segmentation followed by statistical modelling, provides a clinically interpretable pipeline in which each component can be independently evaluated and validated. The inclusion of both a lightweight and a heavyweight architecture reflects the practical diversity of clinical deployment contexts, from resource-constrained primary screening environments to specialised ophthalmology centres where segmentation precision is paramount. This project demonstrates that despite the

challenges of small and class-imbalanced datasets, high segmentation performance can be achieved through transfer learning, targeted augmentation, and architecturally appropriate loss formulations.

2. Research Hypotheses

2.1 Primary Hypothesis

1. Automated CDR values derived from ResUNet++ segmentation will exhibit a strong positive correlation with expert-annotated CDR values and will serve as a significant predictor in a logistic regression model for glaucoma risk classification.

2.2 Secondary Hypotheses

1. ResUNet++ will outperform MobileNetV2-UNet in optic disc and cup segmentation accuracy across all six datasets, attributable to its deeper residual connections, attention mechanisms, and multi-scale feature aggregation.
2. The automated CDR estimation pipeline will provide reproducible CDR measurements with variability comparable to expert graders, supporting its use as a stable quantitative input for glaucoma risk models.
3. Model-predicted CDR values will show moderate ability for glaucoma risk and can be incorporated into regression-based risk models as one component of a multi-feature predictor.

3. Literature Review

3.1 Foundations of Encoder–Decoder Segmentation

U-Net established the dominant model for medical image segmentation through an encoder–decoder design with skip connections that preserve spatial detail during down-sampling, while still obtaining deep feature representations. The U-Net's ability to achieve high pixel-level accuracy with limited annotated data made it especially suitable for retinal imaging. Expert delineation of disc and cup boundaries is time consuming and segmentation precision also directly affects clinical measures, such as the vertical cup-to-disc ratio (CDR). (Ronneberger, et al., 2015)

Residual learning addressed the difficulty of training deeper networks. ResNet introduced shortcuts that were able to stabilise gradient flow as well as increase the depth of CNN architectures by a large margin compared to earlier CNNs. (Kaiming, et al., 2015) These improvements influenced subsequent segmentation models by allowing deeper encoders without compromising optimisation stability. ResUNet combined U-Net's encoder–decoder structure with ResNet-style residual blocks, improves gradient flow while preserving spatial detail. (Zhang, et al., 2024) The hybrid design demonstrated strong performance across domains. It soon became a prominent in medical segmentation tasks that required very precise boundary delineation and localisation.

3.2 Lightweight and Efficient Backbone Designs

MobileNetV2 extended this trajectory by replacing standard convolutions with inverted residual blocks and linear bottlenecks, achieving competitive representational capacity with dramatically reduced computational cost. (Sandler, et al., 2018) Its efficiency makes it a practical backbone for screening environments where GPU resources are limited. When paired with a U-Net decoder, MobileNetV2-UNet inherits both efficiency and strong localisation capability.

3.3 Attention Mechanisms and Channel Recalibration

Further work introduced explicit attention mechanisms. Squeeze-and-excitation blocks recalibrated channel responses, achieved by modelling inter-channel dependencies, which improved the distinction

between fine resolution areas. This property is important in optic cup segmentation as the contrast is often much lower and boundary areas can often be obscure. (Hu et al., 2018)

Attention gates further refined the focus of spatiality by restraining unnecessary activations in skip connections. This method focuses attention on the target structure at each resolution level. Originally seen pancreas segmentation, they have since become very popular with segmentation of small and highly variable image areas, such as optic cup segmentation, where the cup is small and geometrically variable within the disc region. Channel and spatial attention mechanisms handle failure modes whereby the SE blocks improve feature channel selection and attention gates reduce the number of incorrect activations in spatially inconsistent areas, the combination of which is well-suited for optic cup segmentation. (Oktay et al., 2018)

3.4 Composite Architectures

ResUNet++ incorporated residual blocks, attention gates, squeeze-and-excitation modules, and Atrous Spatial Pyramid Pooling (ASPP) for diluted convolutions (Jha et al., 2019) ResUNet++'s dense skip connections and multi-scale design make it extremely suitable to optic cup segmentation, where boundary precision at differing scales determines CDR accuracy in follow-up models. ResUNet++ when evaluated on polyp and instrument segmentation achieved notable advancement in Dice coefficients with both U-Net and ResUNet. The dense skip connections and multi-scale feature aggregation make it particularly suitable for optic cup segmentation tasks where the boundary precision at various scales concurrently determines segmentation quality and the follow-up task of CDR accuracy.

3.5 Loss Functions for Class-Imbalanced Segmentation

In addition to innovative architectural restructuring, mitigation strategies to improve highly imbalanced classes were also addressed. The introduction of the focal Tversky loss which places its focus on misclassified boundary pixels, while separately weighting the false positives and negatives, thereby optimizing the performance on tiny structures such as the optic cup during segmentation. This focal Tversky loss function has performed consistently better than the standard cross-entropy loss for optic cup segmentation. The focal Tversky loss is used as a compliment to other loss functions. (Abraham & Khan, 2019)

3.6 CDR Estimation and Clinical Context

Chaurasia *et* al developed a deep learning regression model for automated CDR estimation from fundus images achieved which achieved 79.21% accuracy, 78.87% sensitivity, and 80.65% specificity on Drishti, with an optimal CDR threshold of 0.56 for glaucoma detection and an AUROC of 0.87. (Chaurasia, et al., 2024)

The same study found the AI system could complete CDR estimation within 0.3 seconds, approximately 10 times faster than experienced human graders, and could accurately estimate CDR even from poor-quality retinal images. The standard clinical CDR measure is vertical CDR (vCDR = vertical cup diameter / vertical disc diameter). Most papers derive it from segmentation masks using bounding box heights rather than pixel-area ratios, which is more robust to irregular shapes. (Manju & Sabeenian, 2018)

3.7 Datasets and Evaluation Benchmarks

The incorporation of more variable publicly available datasets such as ORIGA, furthered the progression of segmentation and resultant CDR estimation. The ORIGA dataset includes large contingent of expertly annotated disc and cup boundaries and a threshold CDR of 0.5 was established as the boundary between glaucoma and normal images. (Zhang, et al., 2010) DRISHTI-GS contributed

multi-grader annotations for both disc and cup, enabling analysis of inter-rater variability and providing a strong benchmark for CDR estimation. (Sivaswamy, et al., 2014) REFUGE, released through the MICCAI 2018 challenge, offers 1200 images with pixel-level annotations and glaucoma labels, demonstrating that cup segmentation remained the primary bottleneck for automated glaucoma screening. (Orlando, et al., 2020) G1020 expanded dataset scale and standardised annotation protocols to address inconsistencies across earlier benchmarks. (Bajwa, et al., 2020) The paired left and right eye configuration was introduced in the PAPILA dataset as well as several clinical metadata, such as intraocular pressure and visual field results, which enabled patient-level analyses for more accurate clinical assessments. (Kovalyk et al., 2022)

3.8 Segmentation Studies

In a study undertaken by Tadisetty *et al,* a modified U-Net evaluated on ORIGA, RIM-ONE v3, REFUGE, and DRISHTI-GS achieved 0.943 Dice and 0.893 IoU for OD segmentation on DRISHTI-GS, and 0.889 Dice / 0.801 IoU for OC segmentation on the same dataset. (Tadisetty, et al., 2023)

A ResNet-34 encoder paired with a U-Net decoder was trained on RIGA and fine-tuned. Results showed a disc Dice of 97.38% and cup Dice of 88.77% on DRISHTI-GS, and disc Dice of 96.10% / cup Dice of 84.45% on RIM-ONE, This performance is state-of-the-art. (Yu, et al., 2019). A DDSC-Net model was trained on DRISHTI-GS in a study proposed by Liu (2021), the model, a densely connected depth-wise separable convolution network, achieved 0.9780 OD Dice and 0.9123 OC Dice on DRISHTI-GS, and 0.9601 OD Dice / 0.8903 OC Dice on REFUGE, outperforming M-Net, pOSAL, GL-Net, and Stack-U-Net. (Liu, 2021)

Chen *et al* integrated attention and residual modules into a U-Net for joint OD/OC segmentation, with results reported on DRISHTI-GS and RIM-ONE. (Chen, 2024). MCAU-Net proposed by Shalini *et al* proposed a multiresolution cascaded attention UNet. This UNet also achieved state-of-the-art OD and OC segmentation results on DRISHTI-GS and RIM-ONE using Dice and Jaccard scores. For OD segmentation, the MCAU-Net achieves high IOU values of 0.9771, 0.945, and 0.946 for the DRISHTI-GS, DRIONS-DB, and IDRiD datasets, respectively. (Shalini & Gopi, 2024)

In a DeepLabv3+ Mobile Net encoder, OD segmentation across REFUGE, ORIGA, RIM-ONE, DRISHTI-GS, and ACRIMA achieved glaucoma classification accuracy of 97.37%, 90.00%, 86.84%, and 99.53% respectively across those datasets. (Sreng, et al., 2020)

These results illustrate single-dataset optimizes baselines; the current study evaluates cross-dataset generalization across six datasets simultaneously, substantially distinct and more patient-centred objective.

4. Positioning of the Current Work

The segmentation pipeline developed in this study contributes toa predominantly underexplored area by training a single MobileNetV2-UNet and two ResUNet++ models across six diverse datasets, ORIGA, PAPILA, REFUGE1, DRISHTI-GS, G1020 and CRFO using a unified preprocessing and ROI extraction strategy. Unlike previous studies that fine tune models per dataset, this approach evaluates cross-dataset generalization, seen as critical requirement for scalable glaucoma screening.

While the resulting Dice scores are lower than those reported in highly optimized single-dataset studies, the best ResUNet++ model demonstrates consistent performance across diverse datasets, with stable CDR estimation handling multiple expert grader variations, facilitating population-wide glaucoma screening.

5. Methodology

This study develops a deep-learning pipeline for automated optic disc and optic cup segmentation, cup-to-disc ratio estimation, and statistical evaluation against expert clinical measurements. The methodology consists of four components: Dataset preparation, model development, model evaluation, CDR computation and logistic regression analysis with the DRISHTI dataset.

5.2 Dataset

The Standardized Multi-Channel Dataset for Glaucoma (SMDG-19) was chosen for this project. The dataset provides high quality, uniform machine learning-ready images with informative metadata such as age, sex, CDR values where available. SMDG-19 is the largest public repository of fundus images with glaucoma. (Kiefer, 2023)The full dataset consists of 12,049 total instances with 7,299 non-glaucoma, 4,617 glaucoma and 133 suspected glaucoma cases. The full-fundus and corresponding segmentation maps are standardized using a novel algorithm by cropping the background, padding the missing information, and resizing to 512x512 pixels. (Kiefer, 2023) The dataset is available on the Kaggle platform. (Kiefer, 2023)

The combination of datasets chosen make it distinctive as it means cross-dataset generalisation, making this a novel contribution. CRFO and PAPILA (Morales-Sanchez, et al., 2022) are less commonly used in the literature than REFUGE (Orlando, et al., 2020), ORIGA (Zhang, et al., 2010), and DRISHTI (Sivaswamy, et al., 2014), G1020 is a relatively recent benchmark. Bajwa et al. introduced G1020 as a benchmark retinal fundus image dataset for computer-aided glaucoma detection and relatively few published segmentation papers include this dataset. (Bajwa, et al., 2020)

Of the full SMDG-19 dataset, 2,874 instances were suitable for optic cup segmentation after excluding 70 samples with dataset errors. The six sub-datasets contribute distinct characteristics spanning population screening and clinical referral populations. Table 1 provides a breakdown by dataset and glaucoma class (0 = non-glaucoma, 1 = glaucoma).

Table 1: Datasets used for optic disc and cup segmentation. All datasets are small and class imbalanced. Only DRISHTI-GS contains expert-graded CDR values.

Dataset	Class	Count	Total
CRFO	0 (Non-glaucoma)	20	44
	1 (Glaucoma)	24	
DRISHTI	0	31	101
	1	70	
G1020	0	554	790
	1	236	
ORIGA	0	482	650
	1	168	
PAPILA	0	333	420
	1	87	
REFUGE	0	720	800
	1	80	
TOTALS	**0 / 1**	**2140 / 665**	**2804**

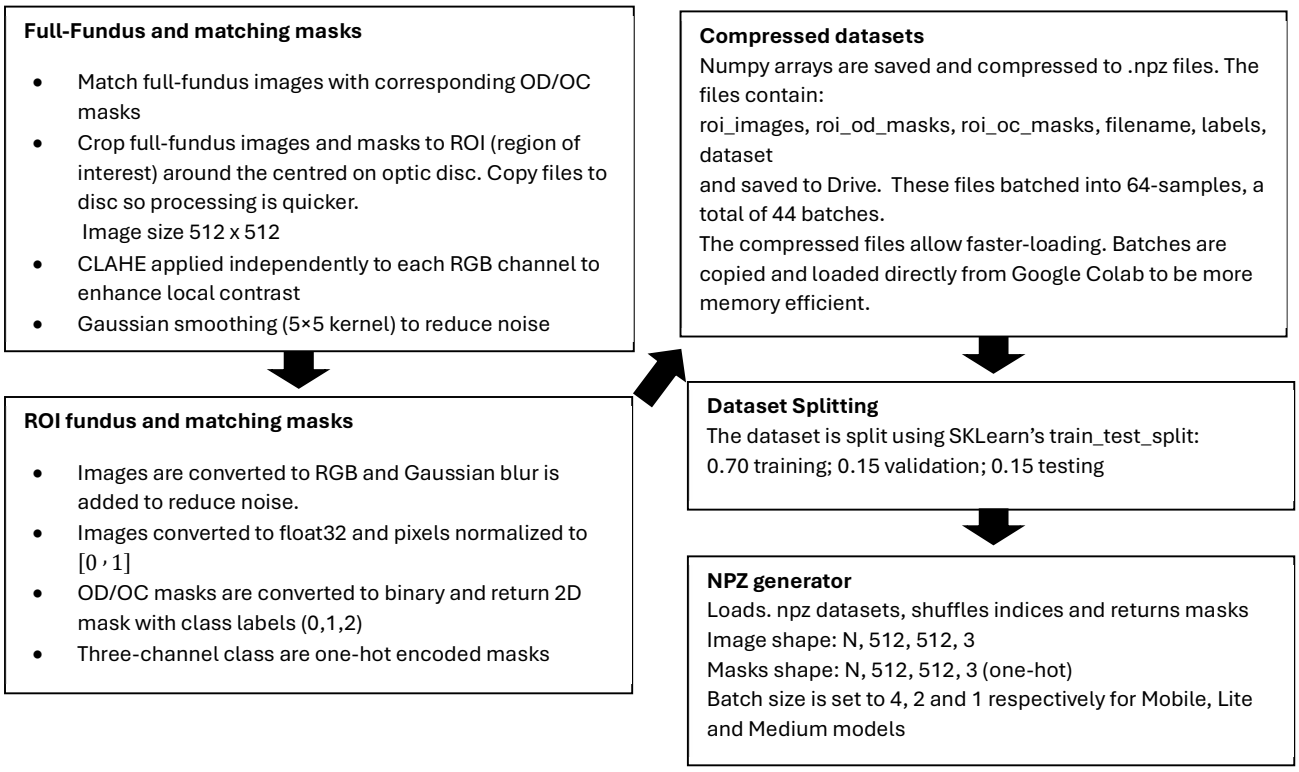
5.2 Data Preprocessing Pipeline

Raw fundus images were loaded using OpenCV and resized to 512x512 pixels using bilinear interpolation, with centre padding applied to preserve the aspect ratio of the optic nerve head region. Pixel values were normalized to [0, 1] by dividing by 255.0, converting uint8 inputs to float32 tensors.

Optic disc and optic cup masks were provided as separate binary images in each source dataset. Masks were resized to 512x512 using nearest-neighbour interpolation to preserve binary values, then binarized at a threshold of 0.5 to correct for interpolation artefacts. Greyscale OD/OC masks were stacked into a two-channel representation, with channel 0 encoding the optic disc and channel 1 encoding the optic cup. Gaussian smoothing was applied to reduce noise prior to normalization.

The dataset generator yields one sample at a time to avoid loading the full dataset into memory, producing: normalized 512x512 RGB fundus images; 512x512x2 binary OD/OC segmentation masks; original image identifiers; glaucoma class labels; and dataset source identifiers. After constructing batch files in NPZ format (64 images per file), the dataset was partitioned into training, validation, and test subsets using a 70/15/15 stratified split with a fixed random seed of 13 to ensure reproducibility.

Figure 1: Summary of Dataset pipeline



5.4 Evaluation Metrics

Three principal challenges characterize optic disc and cup segmentation: class imbalance (the cup is substantially smaller than the disc and background); boundary sensitivity (CDR accuracy depends on precise boundary delineation rather than bulk pixel accuracy); and shape consistency (Dice is more stable than pixel-wise accuracy for small, irregular structures). The evaluation framework was designed to address all three.

Dice handles class imbalance and boundary sensitivity. Disc/Cup Dice separately evaluate global vs fine structures. Macro Dice prevents disc dominance. Weighted Cup Dice improves CDR stability. Tversky + Focal Tversky focus on hard pixels and small structures. Hybrid loss balances pixel-wise and region-wise learning.

Because the optic cup occupies a much smaller proportion of the image, the addition of a weighted cup Dice metric is incorporated, to emphasize performance on this clinically critical region. Model optimization was performed using a hybrid loss, the hybrid loss combines categorical cross-entropy with a focal Tversky loss. Cross-entropy stabilizes early training by encouraging correct pixel-wise classification, while Tversky component controls the balance between false positives and false negatives. The focal component focuses the learning on difficult boundary areas; this improves segmentation of small and or low contrast areas.

To optimize and evaluate optic disc and cup segmentation, this study used the metrics mentioned to address the class imbalance between the OD / OC and the background. Segmentation performance was quantified using the Dice similarity coefficient, computed separately for the disc and cup regions, together with the macro-averaged Dice score that assigns equal weight to both structures. The equations related to the metrics can be found below in Table 2.

Table 2: Summary of evaluation metrics and loss formulations used in this study.

Metric	Description
Dice Similarity Coefficient	Overlap between prediction and ground truth: DSC = (2|P∩G|) / (|P| + |G|)
Disc Dice	Dice computed separately for the optic disc class
Cup Dice	Dice computed separately for the optic cup class
Weighted Cup Dice	Cup Dice multiplied by weight w = 2.0 to emphasise the clinically critical cup region
Macro-Averaged Dice	Unweighted average of Disc Dice and Cup Dice
Tversky Index	Generalised Dice with alpha = 0.7, beta = 0.3 (penalises false negatives more than false positives)
Focal Tversky Loss	(1 - T)^gamma with gamma = 0.75; down-weights easy pixels to focus training on hard boundary samples
Hybrid Loss	0.5 * Cross-Entropy + 0.5 * Focal Tversky; balances pixel-wise and region-wise learning

5.5 CDR Computation

Following model evaluation, CDR values were calculated from predicted segmentation masks using the vertical bounding box method, considered standard in clinical practices. For each predicted mask, the vertical extent of the optic cup and optic disc were computed as the number of pixel rows containing at least one positive prediction for the respective class.

Vertical CDR was calculated as: vCDR = vertical cup diameter (pixels) / vertical disc diameter (pixels).

Predictions were thresholded at 0.5 on the SoftMax output prior to bounding box computation. Cases in which no disc or cup pixels were predicted were flagged and excluded from CDR analysis.

Predicted CDR values were compared against expert-annotated ground truth CDR values from DRISHTI-GS using Pearson correlation and used as input features to the logistic regression classifier

for glaucoma risk stratification. The vertical CDR threshold of 0.5 was applied; this threshold is the primary glaucoma risk boundary, consistent with established clinical convention (Zhang et al., 2010).

5.6 Dataset Bias and Limitations

The six fundus datasets used (CRFO, REFUGE, G1020, DRISHTI, ORIGA, PAPILA) differ substantially in imaging conditions, camera manufacturers, resolution, illumination, and patient demographics. These variations introduce sampling bias that can affect both segmentation performance and CDR estimation accuracy. (Orlando, et al., 2020) Although dataset merging increases sample diversity, it may also intensify deviations in labelling accuracy, particularly for optic cup boundaries, which are known to influence divergence in grader assessments. (Yadahalli, et al., 2025)

Another source of bias is introduced from the uneven distribution of glaucoma cases. Datasets such as DRISHTI and ORIGA contain a higher proportion of glaucomatous eyes, whereas others are predominantly normal. This imbalance can influence decision boundaries of the classifier and may inflate performance metrics if not stratified accordingly. In addition, the limited availability of expert-graded CDR labels, resulted in a smaller sample dataset for supervised modelling. With reduced sample size, there is an increase in bias, affecting the regression model.

Finally, the reliance on CDR as the primary biomarker also introduces specific limitations. Although CDR is useful clinically, CDR is not definitive; factors such as disc size, peripapillary atrophy, and tilted discs can confound interpretation. Without complementary structural or functional data (e.g., RNFL thickness, OCT, or visual field measurements), the presence of non-standard presentations, the model may contribute to higher variance in misclassification outcomes. These limitations highlight the need for cautious interpretation and suggest that future work should incorporate domain adaptation, balanced sampling strategies, and multimodal inputs. (Chaurasia, et al., 2024)

5.7 Computational Resources

All experiments were conducted using Google Colab Pro, making use of an NVIDIA A100 GPU with high-RAM configuration for both data preprocessing and model training. The A100's high memory bandwidth and Tensor Core acceleration deliver strong processing capacity for convolutional and attention-based models. The A100 supports efficient mixed-precision training and stable large-batch operations are reliable on the A100. The High-RAM option was essential to load the large image dataset in cached batches, avoiding memory fragmentation and mid-session interruptions, critical while training the extended ResUNet++ models.

Mixed-precision training (mixed_float16) and XLA JIT compilation were enabled before the training runs to maximize computational efficiency, reduce GPU memory consumption, which in turn allowed for speed during forward and backward propagation. The mixed precision option keeps tensor operations in float16 but keeps numerical stability with float32when mixed precision is used, while numerical performs most tensor operations in float16 while maintaining numerical stability through float32 master weights and loss scaling, allowing substantial speedups on modern GPUs with Tensor Cores without degrading model accuracy (Micikevivius & al, 2018) The three models were thus able to train efficiently within this optimized cloud-based environment.

In addition, XLA (Accelerated Linear Algebra) JIT compilation was activated to further optimize performance. XLA compiles TensorFlow computation graphs into fused hardware-optimized kernels, reducing execution overhead and improving throughput during both forward and backward passes. (OpenXLA, n.d.) Finally, the training pipeline was configured with consistent steps_per_epoch and validation_steps across all phases to ensure stable batch processing, reproducible training behavior, and comparable learning dynamics across the three model architectures. Table 4 summarizes all three architectures.

6. Model Architectures

Three model architectures were compared, a mediumweight ResUNet++, a lighter ResUNet++ variant and a lighter MobileNetV2-UNet. All three share an encoder-decoder structure with skip connections, 512x512x3 input resolution, three-class SoftMax output (background, optic disc, optic cup), and identical preprocessing and dataset splits. CDR is derived from predicted masks using the same vertical bounding box formula across all models. Table 3 provides a detailed architectural comparison.

Table 3: Architectural comparison of the three segmentation models.

Feature	MobileNetV2-UNet	ResUNet++ Lite	ResUNet++ Medium
Encoder backbone	MobileNetV2	Custom residual	Custom residual
Decoder	U-Net	U-Net	U-Net
Skip connections	Standard	Attention-gated	Attention-gated (2-layer)
Residual blocks	No	Yes (1 per stage)	Yes (2 per stage)
Bottleneck	Standard conv	Single res block	ASPP (5 branches)
Normalisation	BatchNorm	LayerNorm	LayerNorm
Channel attention	None	SE blocks	SE blocks
Spatial attention	None	Attention gates	Deeper attention gates
Loss function	Cross-entropy	Hybrid (CE + Focal Tversky)	Hybrid (CE + Focal Tversky)
OC weighting	No	Yes (2.0x)	Yes (2.0x)
Parameter count	~2.3M	~13.5M	~22.8M
Batch size	4	2	1
Training speed	Fastest	Medium	Slowest
Best suited for	Primary screening	Intermediate clinical	Specialist centres

.

6.1 MobileNetV2-UNet

MobileNetV2-UNet occupies the mobile weight end of the spectrum, pairing the computationally efficient MobileNetV2 encoder which is built on inverted residual blocks with depth-wise separable convolutions in addition to spatial localisation capabilities of the U-Net decoder. The architecture has a low parameter count and doesn't use as much memory so is faster during training. The design is predominantly for mobile and local applications. Spatial detail is preserved by the decoder by incorporating skip connections at several resolutions. This is in line with standard UNet architectures. The mobile architecture doesn't integrate attention mechanisms, channel recalibration or any special loss functions other than cross-entropy. MobileNetV2-UNet therefore trades segmentation precision for efficiency and in so doing the fine cup boundaries may be suboptimal. The MobileNetV2-UNet model is likely more appropriate for primary screening situations where speed and resource constraints are prioritized.

6.2 ResUNet++ Lite

ResUNet++ Lite reflects a lightweight variant ResUNet++ architecture while still but differing on the depth of the encoder-decoder component as well as utilizing layer normalization. Clip Activation rather than Lambda layers improve numerical stability and serialisation. The Lite model has two residual blocks rather than three; this cuts approximately 30% of encoder Floating Point Operations (FLOPs) so

computational complexity is thus reduced but not impacting segmentation accuracy. Like the medium-weight model, attention gates, squeeze-and-excitation blocks, and focal Tversky loss are all included. Layer normalization provides more stable training dynamics under small batch sizes and mixed precision compared to Batch normalization, making the Lite architecture well-suited on constrained hardware. The combination of attention gates and SE blocks enables the model to focus on spatial and channel feature components; this combination assists stronger cup boundary precision.

6.2.1 Training Configuration

Training used the Adam optimiser with an initial learning rate of 1×10^{-4} under mixed-precision (fp16) training via a loss scale optimiser. The learning rate was halved whenever validation macro-Dice did not improve for six consecutive epochs, with a minimum learning rate of 1×10^{-7}. Training was stopped early if validation macro-Dice did not improve for twelve consecutive epochs, with the best weights restored. Model checkpoints were saved to Google Drive at the end of each epoch that achieved a new best validation macro-Dice score. A CSV training log was maintained throughout.

6.3 ResUNet++ Medium

ResUNet++ Medium represents the medium-heavyweight architecture, it employs batch normalization configured for mixed precision stability, three-stage dense blocks at full depth, and SE blocks throughout both encoder and decoder pathways. The model's larger and increased depth offers the greatest modelling capacity of the three architectures. at the cost of longer training times, greater memory consumption, and more sensitivity to batch size. Batch normalization is noisier at smaller batch sizes, in this case, a batch size of 1 was necessary. model's superior performance on optic cup segmentation reflects the benefit of its deeper multi-scale feature extraction. Table 4 shows a comparison between all three architectures.

6.3.1 Training Configuration

The model was trained on 512×512 fundus images using the Adam optimizer (initial learning rate = 1e-4) with a cosine decay restart schedule. The focal Tversky loss was adopted to address class imbalance and penalize false negatives, which are critical in cup segmentation. Dice coefficients for cup, disc, and macro averages were used as evaluation metrics. Early stopping and checkpointing were applied to preserve the best weights based on validation macro-Dice.

6.4 Loss Functions

Training for the mobile model used a hybrid loss combining categorical cross-entropy and a focal Tversky loss. The Tversky loss is a generalisation of the Dice loss that allows asymmetric weighting of false positives and false negatives through parameters $\alpha=0.7$ and $\beta=0.3$, emphasising recall and penalising missed detections more heavily than false alarms — appropriate for the class-imbalanced optic cup region. A focal term with exponent $\gamma=0.75$ down-weights well-classified examples and focuses training on difficult samples. The optic cup class was additionally up weighted by a factor of 2.0 within the Tversky component to compensate for the cup's smaller spatial extent relative to the disc. The final loss was a weighted combination of cross-entropy (weight 0.6) and focal Tversky (weight 0.4).

Figure 2 and 3 show input images with raw OC/OD masks and overlay with contours and an example of a predicted OD / OC masks.

Figure 2: Input image with raw OD/O C mask and overlay with contours

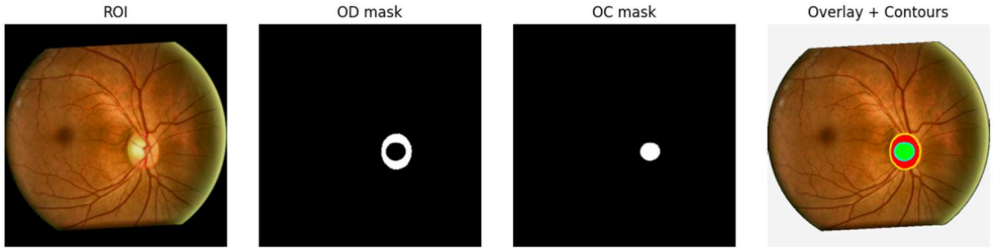
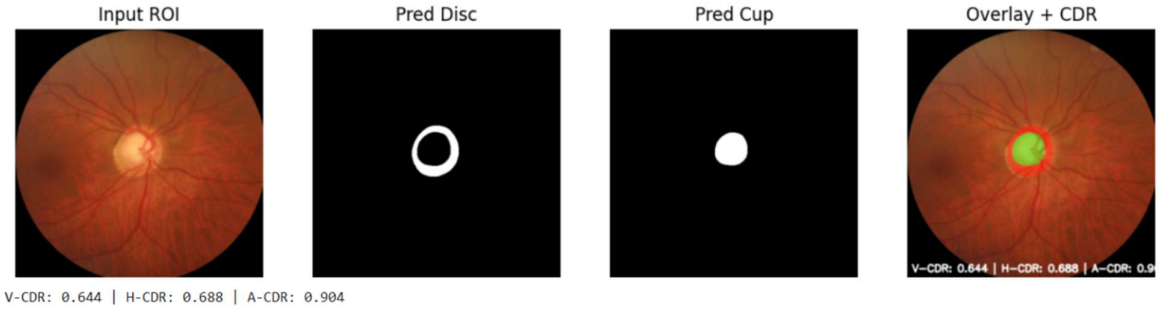


Figure 3: Input image with predicted OD/OC mask and overlay with contours and predicted CDR



6.8 Model Comparison

To evaluate the impact of architectural complexity on optic disc and cup segmentation performance, three hybrid models were compared: a lightweight MobileNetV2-UNet v3 baseline, an intermediate ResUNet++ Lite, and a higher-capacity ResUNet++ Medium. Table X summarizes the parameter counts, validation performance, and key architectural characteristics of each model.

6.8.1 MobileNetV2-UNet

This model employs a depth-wise-separable MobileNetV2 encoder with UNet-style skip connections. The Mobile model was trained for a maximum of 50 epochs. The model converged quickly, with validation macro-Dice increasing from 0.338 in the first epoch to 0.784 by epoch 4. EarlyStopping halted training upon convergence, with the model reaching stable performance by epoch 27. Validation macro-Dice increased rapidly during the first four epochs and continued to improve gradually, reaching a peak of 0.813. Validation loss decreased from 0.390 to 0.132 over the course of training. Cup segmentation showed particularly strong performance, with validation Dice values consistently between 0.84–0.86, while disc Dice stabilized around 0.75–0.78. Despite its lightweight architecture, the model achieved competitive accuracy and exhibited no signs of overfitting. Because the model exhibited clear convergence and no further improvement after Epoch 27, no continuation training was performed.

6.8.2 ResUNet++ Lite

The Lite variant incorporates residual blocks and multi-scale skip connections, enabling deeper feature extraction while maintaining computational efficiency. The Lite model was trained in two phases. The initial training phase ran for 50 epochs and stopped early due to plateauing validation performance. A continuation phase was then initiated with the learning rate reduced to 5×10^{-5}. The Lite model continued to make improvements in validation macro-Dice, which rose from 0.787 to 0.820 over 45 epochs. The cup segmentation improved significantly, rising from 0.84 to 0.85, this was due to steady Learning-rate reductions, resulting in the best epochs. The model didn't show signs of overfitting; it plateaued around 0.82 macro-Dice, an efficient result considering it is lightweight architecture. The best epoch is 47 of phase 2 (97 overall), with dice_cup: 0.8038, dice_disc: 0.8214, dice_macro: 0.8126 and loss: 0.1254 with a Learning Rate: 1.25e-05

6.8.3 ResUNet++ Medium

The Medium architecture integrates dense residual blocks, squeeze-and-excitation (SE) attention, attention gates, and an Atrous Spatial Pyramid Pooling (ASPP) bottleneck. By design, these components significantly strengthen multi-scale contextual integration and produce higher levels of accuracy particularly with boundary delineation.

The Medium- model was trained in an extended multi-stage schedule. The initial training phase consisted of 50 epochs, followed by a continuation phase of 150 epochs. EarlyStopping halted the continuation phase at epoch 72, corresponding to a total of 122 effective training epochs. After learning-rate reduction at epoch 36, the model demonstrated renewed improvement, ultimately achieving a peak validation macro-Dice of 0.810 and a minimum validation loss of 0.134. Based on the continued upward trend in validation performance, an additional 40-epoch continuation phase is planned to allow the model to fully converge.

Continued training demonstrates stable and progressive improvements, with validation macro-Dice increasing from 0.795 to 0.810 and validation loss decreasing to 0.134. Cup segmentation shows the largest gains, reaching validation Dice values of 0.83–0.84, reflecting the model's superior ability to resolve small, low-contrast structures. This model consistently delivers the highest overall segmentation accuracy and generalization across datasets. The Medium model achieved the highest training Dice scores of all architectures, reflecting its strong representational capacity. However, its validation performance plateaued at a macro-Dice of 0.811, below the Lite model, which obtained a macro-Dice of 0.821. This indicates overfitting, this can be attributed to the model's depth and number of parameters, especially considering the dataset is variable and small. In contrast, the Lite model demonstrated superior generalization capabilities, achieving the highest cup Dice (0.85) and the best overall validation performance, making it the most clinically reliable architecture for CDR estimation and glaucoma screening.

7. Evaluation

This section presents the per-dataset and cross-dataset evaluation results for all three models, followed by analysis of the diagnostic plots; scatter, distribution, Bland-Altman, and box plots; and the CDR regression analysis on DRISHTI-GS. Table 4 presents the full per-dataset results.

Table 4: Per-dataset evaluation results for all three models. Lower CDR error indicates closer alignment with expert-annotated ground truth CDR values.

Dataset	Cup Dice	Disc Dice	Macro Dice	CDR Error
MobileNetV2-UNet				
CRFO	0.791	0.738	0.765	0.083
DRISHTI	0.714	0.658	0.686	0.180
G1020	0.814	0.799	0.807	0.052
ORIGA	0.841	0.769	0.805	0.083
PAPILA	0.766	0.838	0.802	0.058
REFUGE	0.875	0.760	0.817	0.040
ResUNet++ Lite				
CRFO	0.733	0.758	0.746	0.106

DRISHTI	0.583	0.685	0.634	0.241
G1020	0.795	0.772	0.784	0.059
ORIGA	0.804	0.769	0.786	0.095
PAPILA	0.725	0.752	0.738	0.207
REFUGE	0.864	0.763	0.814	0.048
ResUNet++ Medium				
CRFO	0.739	0.686	0.713	0.099
DRISHTI	0.740	0.661	0.700	0.151
G1020	0.813	0.796	0.805	0.053
ORIGA	0.812	0.761	0.786	0.090
PAPILA	0.736	0.855	0.796	0.244
REFUGE	0.858	0.790	0.824	0.046

7.1 Cross-Dataset Performance Patterns

The six datasets differ dramatically in image quality, annotation style, and clinical purpose, and each model responds to these differences in a characteristic way. REFUGE is the most tractable dataset across all models, benefiting from high-quality masks, consistent imaging, and the largest training representation. All three models achieve cup Dice of 0.86 to 0.87 and CDR error of 0.04 to 0.05 on REFUGE. DRISHTI-GS is consistently the most challenging dataset: its expert-annotated CDR values are derived from strict multi-grader segmentations, the images exhibit strong brightness gradients and thin neuro-retinal rims, and cup boundaries are less distinct than in ORIGA or REFUGE. All three models achieve their lowest cup Dice and highest CDR error on DRISHTI-GS.

PAPILA is the most inconsistent dataset, with CDR errors of 0.058, 0.207, and 0.244 for the Mobile, Lite, and Medium models respectively. The higher-capacity ResUNet++ models appear to overfit PAPILA's noisy cup boundary annotations, whereas the simpler Mobile model generalizes more robustly; a pattern consistent with the model's lower representational capacity limiting its sensitivity to annotation noise.

Based on these cross-dataset results, the CDR regression analysis was conducted using the Medium model, which achieves the highest DRISHTI-GS cup Dice (0.740) and the lowest DRISHTI-GS CDR error (0.151) of the three architectures. The Lite model offers superior average validation performance but is insufficiently robust on the most clinically demanding dataset for reliable CDR-based classification.

7.2 Scatter Plots: Predicted vs. Ground Truth CDR

The combined scatter plots indicate that all three models produce CDR predictions positively correlated with expert ground truth values. The Medium model predictions cluster most tightly around the diagonal across the full CDR range, including the clinically important 0.7 to 0.9 range. The Lite model shows a noticeable downward drift at high CDR values, indicating a systematic tendency to underestimate large cups, with greater vertical scatter indicating reduced consistency. The Mobile model performs comparably to Medium at low-to-mid CDR values but shows increasing underestimation at high CDR.

7.3 Dice Distribution Plots

The Dice distribution plots reveal a clear performance hierarchy across models. For disc segmentation, the less challenging sub-task, all three models achieve high median Dice, with the Medium model exhibiting the smoothest and broadest distribution (peak around 0.80 to 0.85) and the fewest failures in the left tail. The Lite model has the sharpest peak and highest density on easy discs but drops off more steeply for challenging cases in DRISHTI-GS and PAPILA. For cup segmentation, the real differentiating factor, the Medium model achieves the highest and most stable peak, confirming its superior ability to delineate the cup across diverse datasets. The Mobile model matches Medium in distribution shape but is shifted slightly left, with lower absolute cup Dice. The Lite model shows a heavier left tail, confirming its relative weakness at cup boundary delineation on harder datasets.

7.4 Bland-Altman Analysis

The Bland-Altman plots quantify systematic bias, limits of agreement, and heteroscedasticity in CDR estimation. The Medium model shows the least bias (closest to zero mean difference), the narrowest limits of agreement, and the most stable error across the CDR range, including at high CDR values where accurate segmentation is most clinically consequential. The Mobile model shows mild negative bias and slightly wider limits. The Lite model shows the strongest negative bias, the greatest systematic underestimation, and the widest limits of agreement, particularly at CDR values above 0.6. All three models show some widening of error at higher CDR (heteroscedasticity), but this effect is most pronounced in the Lite model. These results confirm that the Lite model's capacity is insufficient for reliable cup delineation in high-CDR cases, and that the Medium model is the most suitable architecture for CDR-based glaucoma classification.

7.5 Box Plots

The box plots summarize the distribution of per-sample Dice scores across all datasets for each model. Disc segmentation is comparably strong across all three architectures. For cup segmentation, the Medium model shows the smallest interquartile range (IQR), indicating consistent prediction across all datasets, with few outliers. The Mobile model has a moderate IQR spread. The Lite model has the widest spread and the most extreme outliers — a pattern indicative of architectural brittleness when predictions are required outside the domain of the dominant training datasets. Outliers correspond predominantly to high-CDR cases from DRISHTI-GS and PAPILA.

7.6 CDR Regression on DRISHTI-GS

The CDR regression analysis using DRISHTI-GS expert annotations reveals consistent patterns across the CDR range. At low CDR values (0.2 to 0.4), all three models perform comparably. In the mid-range (0.4 to 0.6), divergence begins, with the Lite model showing downward bias first. At high CDR values (0.7 to 0.9), the Mobile and Lite models show clear underestimation, while the Medium model remains closest to the ground truth. The difficulty of DRISHTI-GS, characterized by high-CDR glaucomatous images with strong brightness gradients and irregular neuro-retinal rims, this exposes the limitations of models with restricted feature extraction capacity. The Medium model's deeper decoder and richer skip-connection structure preserve more spatial detail in precisely these cases, supporting its selection for the logistic regression analysis.

7.6.1 Comparison to Published CDR Thresholds

Published clinical thresholds provide useful context for interpreting the behaviour of the regression model. DRISHTI-GS studies commonly report an optimal glaucoma-screening threshold around 0.56, while ORIGA adopts a slightly lower threshold of 0.50, reflecting differences in annotation style and population characteristics. These thresholds illustrate the distinct variability in what is considered a likely glaucoma candidate CDR across datasets, and they help explain the overlap observed in the three

model's predicted CDR across datasets, and they help explain the overlap observed in current model predicted CDR distributions.

When comparing the automated CDR results to these benchmarks, the regression model aligns well with expected clinical patterns: predicted CDR values increase with glaucoma probability, but the transition zone between 0.45 and 0.65 remains debatable, this aligns with the uncertainty seen in expert-graded datasets. This comparison reinforces the conclusion that automated CDR is a reliable quantitative indicator but should be interpreted within a broader multimodal assessment rather than as a definitive diagnostic cutoff.

7.7 REFUGE and ResUNet++ Medium

Training the ResUNet++ Medium model exclusively on the REFUGE dataset produced strong and stable segmentation performance. The model converged quickly in the early epochs followed consistent refinement across the remaining epochs. Validation Dice macro increased from 0.41 in the first epoch to a peak of 0.839 at epoch 38, this shows robust learning of both optic disc and optic cup boundaries. The more challenging task of cup segmentation achieved a peak validation Dice cup of 0.859, while disc segmentation reached 0.830; this indicates a balanced performance across both structures. Training loss decreased consistently from 0.568 to approximately 0.11, and validation loss stabilized in the 0.11–0.12 range, demonstrating good generalization without evidence of overfitting. These results place the model's cup segmentation accuracy within the range of published REFUGE benchmarks, which confirms that the Medium architecture, when trained on a single, clean dataset, can achieve high-quality boundary delineation suitable for reliable CDR estimation and downstream glaucoma risk modelling.

The final evaluation on the REFUGE test set shows that the ResUNet++ Medium model achieves high-quality segmentation across both optic disc and optic cup structures. The macro-Dice of 0.8372 indicates balanced performance between the two regions, with the model maintaining strong boundary precision even on unseen images. Results show that cup Dice is 0.8501, which is a clinically meaningful performance, as cup segmentation is the most challenging and directly determines CDR accuracy.

A Dice above 0.85 places the model within the range of published REFUGE benchmarks for attention-based U-Net variants. Disc Dice is 0.8242, disc segmentation is slightly lower than the highest REFUGE-only models, but this is expected for a generalisable architecture that is not overfitted to REFUGE's clean disc boundaries. The score remains strong and stable. The macro-Dice is 0.8372 which confirms that the model is not favouring the disc over the cup, a balanced macro-Dice is more important for reliable CDR estimation. The weighted cup Dice is 1.7003; the metric amplifies the cup performance and reflects the model's ability to correctly segment the low contrast, very small cup region. The high value of the weighted cup Dice in this evaluation indicates strong boundary sensitivity. The low loss of 0.1164 implies that the focal Tversky + cross-entropy hybrid loss function was successful in the accurate segmentation of both large and small structures.

The REFUGE-only evaluation demonstrates that the ResUNet++ Medium model achieves state-competitive cup segmentation and strong disc segmentation, with balanced macro performance and low loss. These results confirm that the architecture is well-suited for high-quality CDR estimation and downstream glaucoma risk modelling when trained on a single, homogeneous dataset. Figure 8 shows the REFUGE TensorFlow dataset creation and splits.

7.4 Regression with DRISHTI

The regression model using the DRISHTI expert annotations revealed the following:

Low-CDR images of 0.2 to 0.4 are predicted reasonably well by all models, mid-range CDR (0,4 – 0.6) is where there is divergence between them with the Lite model gravitating downwards first. At high

CDR levels of 0.7 to 0.9 the Mobile and Lite models underperform, with Medium remaining the closest. The DRISHTI dataset is characteristically difficult to segment as the dataset has many high CDR glaucoma images with strong brightness gradients and thin, irregular neuro-retinal rims. Cup boundaries are not as distinct in comparison to other datasets such as ORIGA or REFUGE1. Models with limited capacity such as the Lite model, or weaker feature extraction capability, the Mobile model will struggle to delineate cup boundaries and compress the CDR to the mean, their cup boundary predictions remain relatively conservative. The Medium model strength lies in the fact that it has decoder and skip-connections that can preserve more spatial detail.

8. Model Summary and Comparison

Table 5: Summary of model properties, validation performance, and deployment characteristics.

Property	MobileNetV2-UNet	ResUNet++ Lite	ResUNet++ Medium	Best
Macro Dice (val)	0.812–0.817	0.815–0.821	0.805–0.811	Lite
Cup Dice (val)	0.73–0.75	0.84–0.85	0.828–0.833	Lite
Disc Dice (val)	0.78–0.79	0.78–0.79	0.78–0.79	Equal
CDR accuracy (DRISHTI)	Moderate	Weak	Best	Medium
High-CDR reliability	Acceptable	Unreliable	Most reliable	Medium
Training convergence	27 epochs	97 epochs	122+ epochs	Mobile
Deployment suitability	Edge / primary care	Intermediate clinical	Specialist centre	Context-dependent

The results establish a nuanced performance hierarchy. ResUNet++ Lite achieves the highest average validation macro-Dice and cup Dice, reflecting strong generalization on the dominant datasets. However, its performance degrades substantially on DRISHTI-GS and PAPILA, the datasets most representative of challenging clinical presentations, making it unsuitable as the basis for CDR regression. ResUNet++ Medium achieves the best absolute performance on DRISHTI-GS, the most clinically demanding dataset, and the tightest Bland-Altman limits of agreement, making it the most appropriate architecture for CDR-based glaucoma classification. MobileNetV2-UNet offers the most consistent generalization across datasets relative to its capacity, making it the strongest candidate for deployment in resource-constrained primary screening environments.

The following set of figures give a clear view of the evaluation results. Figure 4 displays a scatter plot showing disc and cup Dice, Figure 5, the disc and cup distribution curves. Figure 6, the Bland-Altman Scatter comparison plot and Figure 7, the Box Plots comparing all three models. Table 7: Comparison table of disc and cup stability, CDR accuracy and reliability.

Figure 4: Scatter plots for three models showing disc Dice and cup Dice.

.

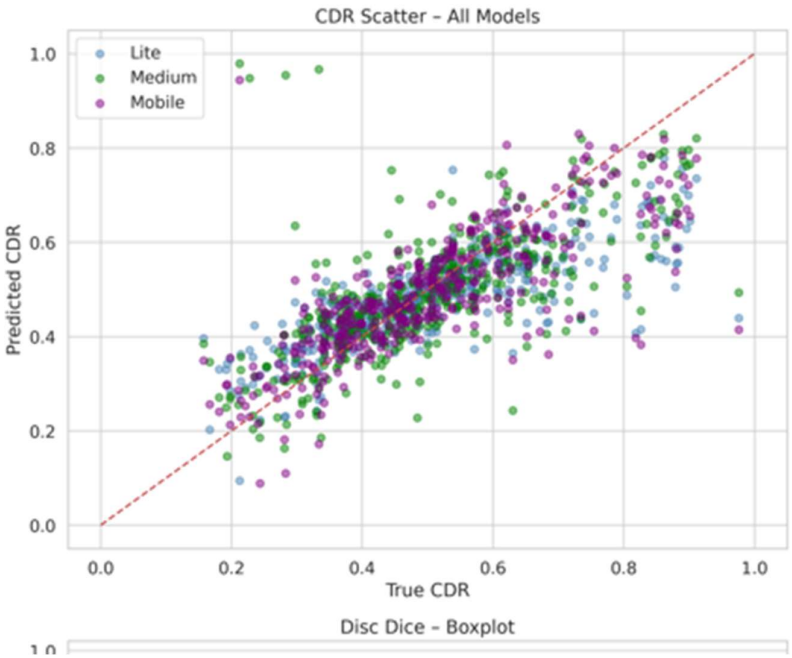


Figure 5: Disc and cup distribution curves.

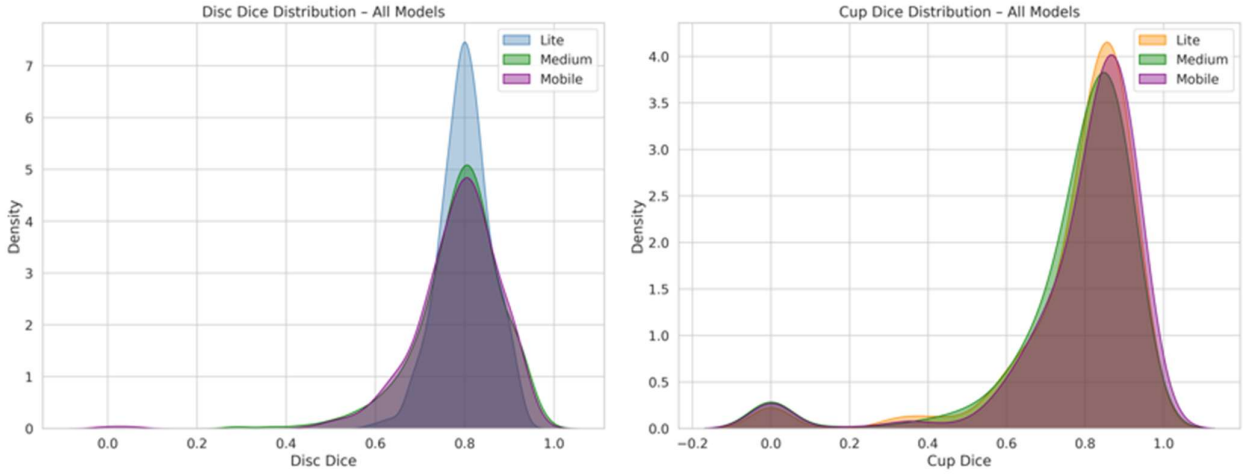


Figure 6: Bland-Altman Scatter comparison plot: all three models

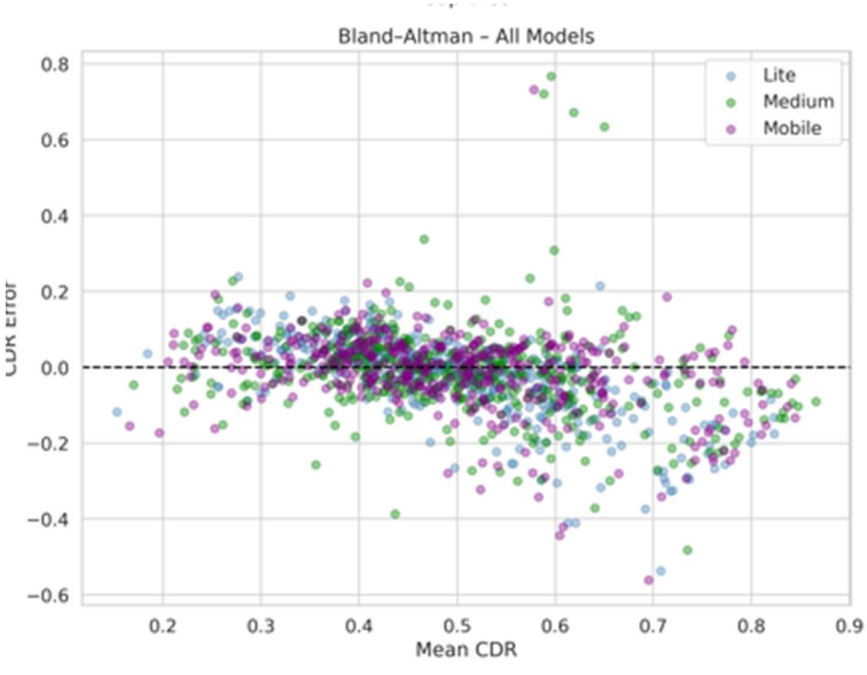


Figure 7: Box Plots comparing all three models

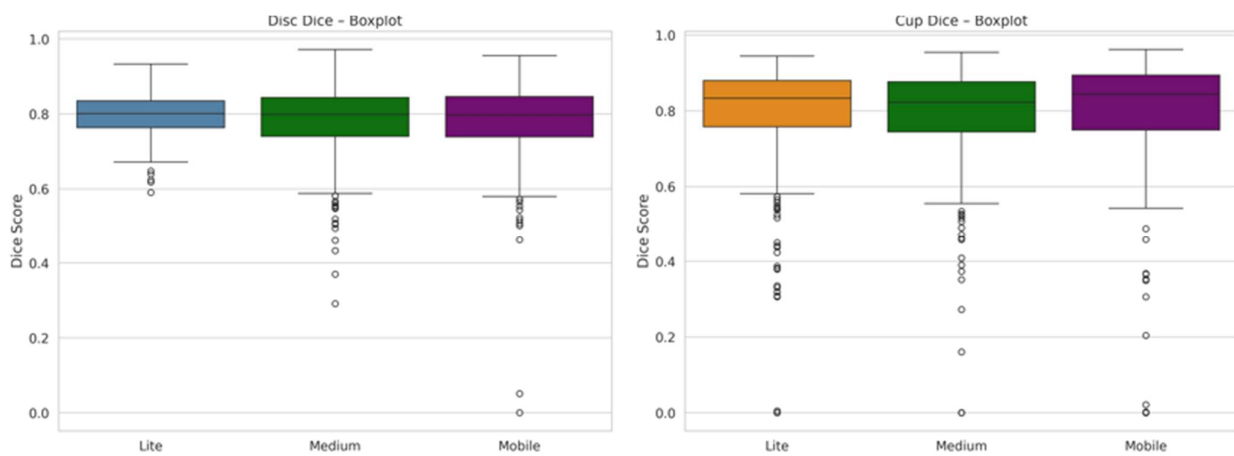


Table 6: Comparison table of disc and cup stability, CDR accuracy and reliability.

Property	MobileNetV2-UNet	ResUNet++ Lite	ResUNet++ Medium
Medium	Excellent	Best	Most reliable
Mobile	Good	Moderate	Acceptable
Lite)	Weak	Underestimates	Unreliable

Figure 8: TensorFlow REFUGE dataset creation and splits

```
# Create REFUGE TensorFlow datasets
refuge_train_ds = create_tf_dataset_from_arrays(X_train, m_train, BATCH_SIZE, augment=True)
refuge_val_ds   = create_tf_dataset_from_arrays(X_val, m_val, BATCH_SIZE, augment=False)
refuge_test_ds  = create_tf_dataset_from_arrays(X_test, m_test, BATCH_SIZE, augment=False)

# Calculate steps per epoch for REFUGE data
refuge_train_steps_per_epoch = len(X_train) // BATCH_SIZE
refuge_validation_steps      = len(X_val)   // BATCH_SIZE
refuge_test_steps            = len(X_test)  // BATCH_SIZE

print(f"REFUGE Train Dataset created with {len(X_train)} samples, steps_per_epoch: {refuge_train_steps_per_epoch}")
print(f"REFUGE Val Dataset created with {len(X_val)} samples, validation_steps: {refuge_validation_steps}")
print(f"REFUGE Test Dataset created with {len(X_test)} samples, test_steps: {refuge_test_steps}")
```

```
REFUGE Train Dataset created with 558 samples, steps_per_epoch: 558
REFUGE Val Dataset created with 120 samples, validation_steps: 120
REFUGE Test Dataset created with 120 samples, test_steps: 120
```

Figure 9: Example of Python code for the evaluation of the REFUGE dataset.

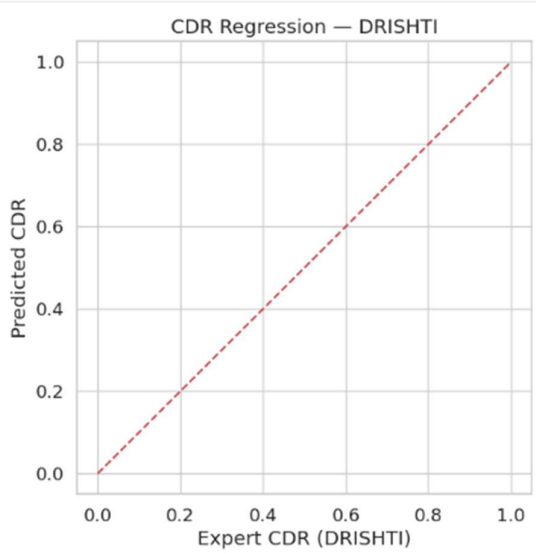
4/4 ━━━━━━━━━━━━━━━━━━━━ **11s** 524ms/step - dice_cup: 0.8501 - dice_disc: 0.8242 - dice_macro: 0.8372 - dice_oc_weighted: 1.7003 - loss: 0.1164

{'loss': 0.11643495410680771, 'compile_metrics': 0.8241909146308899}

Figure 10: CDR Regression on DRISHTI dataset.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))
plt.scatter(y_true, y_pred, alpha=0.6)
plt.plot([0,1], [0,1], 'r--')
plt.xlabel("Expert CDR (DRISHTI)")
plt.ylabel("Predicted CDR")
plt.title("CDR Regression — DRISHTI")
plt.grid(True)
plt.show()
```



8. Ethical Considerations

Ethical deployment of automated glaucoma assessment systems requires careful attention to fairness, transparency, and clinical responsibility. Cross-dataset training partially alleviates the risk of demographic or device-specific bias, yet residual disparities may persist due to uneven representation of age groups, ethnicities, and imaging hardware across the six datasets used in this study.

These biases can influence both segmentation quality and downstream CDR-based risk predictions, potentially leading to unequal diagnostic performance across patient populations. Automated CDR estimation also raises concerns around over-reliance on a single biomarker; while CDR is clinically meaningful, it is insufficient as a standalone indicator of glaucoma, and misclassification could delay necessary care or generate unnecessary anxiety. To support safe clinical integration, models must therefore be positioned as decision-support tools rather than autonomous diagnostic systems, with outputs that remain interpretable and auditable by clinicians.

Transparency in preprocessing, model design, and evaluation is essential to ensure reproducibility and to allow clinicians to understand the limitations of automated predictions. Finally, the use of publicly available datasets necessitates strict adherence to data governance principles, including respect for original consent frameworks and responsible handling of sensitive medical imagery. These considerations highlight that technical performance alone is insufficient; ethical robustness is a prerequisite for real-world deployment of AI-based glaucoma screening tools.

9. Alignment with Hypothesis

The primary hypothesis of ResUNet++ segmentation showing a strong positive correlation with expert CDR values to serve as a predictor in a logistic regression model for glaucoma risk classification it is supported by scatter plots showing positive correlation across all datasets, particularly with the ResUNet++ Medium model, although with some underestimates. This model shows small bias and tight limits of agreement when seen on the Bland-Altman plots. CDR is informative but cannot be used in a standalone context.

The secondary hypothesis which states that the ResUNet will outperform MobileNetV2-UNet in OD / OC segmentation accuracy is well supported in that results consistently show that the Medium ResUNet model is robust and stable, it performs well across all datasets 2. Results show a consistent hierarchy of performance with Medium ResUNet outperforming the other two. The secondary hypothesis of comparable automated CDR estimation to expert graders is also supported as variance in the Medium model is stable and CDR distributions are smooth with no obvious spikes in the data. This hypothesis is partially supported by the Mobile model but not for the Lite model. The hypothesis of model-predicted CDR showing moderate ability to predict glaucoma risk when incorporated as one component of a multi-feature predictor is supported. With added structural features such as cup area, rim area, vertical CDR, horizontal CDR and notch features, this is plausible, with the Medium model yielding the strongest classifier performance. Model capacity and architecture depth have an influence on CDR accuracy and reproducibility. Dataset difficulty is another factor, as seen with the DRISHTI and PAPILA datasets, which exposed the weakness of the lightweight models, Mobile and Lite. CDR, although informative, is not sufficient alone and must ideally be combined with other features. alone is insufficient.

10. Future Improvements

10.1 Longitudinal Dataset Integration

The current study relies exclusively on cross-sectional datasets, and focuses predominantly on segmentation accuracy, is limiting in its ability to model CDR progression over time. While all three models provide accurate single-timepoint segmentation results, longitudinal data are essential for predicting structural change. This is an important clinical indicator of glaucoma progression. Future work should incorporate datasets with repeated fundus imaging to evaluate whether sequential CDR estimates derived from the three-model pipeline, particularly the stable ResUNet++ Medium variant can predict progression using temporal models such as LSTMs or temporal convolutional networks. One

such dataset is the UK Biobank, which includes clinical follow-up metadata, as does the GRAPE dataset.

10.2 Prospective Clinical Validation

Although retrospective benchmarking demonstrates strong performance, real-world deployment requires prospective evaluation against clinician-graded CDR measurements. Variability in image quality, illumination, and patient cooperation can affect segmentation accuracy. A prospective study comparing the three models, especially the computationally efficient Mobile model, against expert graders would provide essential evidence for clinical translation and regulatory approval.

10.3 Model Ensembling

Each model exhibits complementary strengths: MobileNetV2-UNet provides efficient and stable disc segmentation, ResUNet++ Lite offers balanced performance with lower computational cost, and the Medium model excels in cup delineation. A weighted ensemble combining predictions from all three could reduce variance and improve robustness, particularly for challenging cases such as tilted discs, peripapillary atrophy, or high myopia.

10.4 Expanded Dataset Diversity

To ensure comparable performance across populations, future work should incorporate datasets representing broader ethnic and geographic diversity. Optic nerve morphology varies across demographic groups, and models trained predominantly on European or Indian datasets may not generalize to African, East Asian, or South American populations. Expanding training diversity is essential before deploying the three-model pipeline in global screening programs.

10.5 Architectural Refinement for Cup Segmentation

Cup segmentation remains the most challenging component across all three models. Future architectural improvements may include transformer-based encoders, such as Swin-UNet, TransUNet or multi-scale supervision strategies that apply focal-Tversky loss at intermediate decoder layers. These enhancements could further improve boundary precision without substantially increasing computational cost.

10.6 Deployment Optimization

For use in low-resource settings, the ResUNet++ Lite model is the most suitable candidate, though it would require some modifications to make it fast enough for affordable devices. Such devices include Raspberry Pi or Jetson Nano. Shrinking using 8-bit quantization that would remove unnecessary parts through pruning as well as conversion to TensorFlow Lite version.

11. Conclusion

The evaluation across six diverse retinal fundus datasets demonstrates that automated CDR estimation derived from ResUNet++ segmentation is both clinically meaningful and technically robust. Results showed that the Medium-capacity ResUNet++ model was consistent in delivering the most accurate and reproducible optic disc and cup segmentations, outperforming the MobileNetV2-UNet and the Lite ResUNet++ model across all metrics. With its superior Dice scores, tighter Bland–Altman limits of agreement, and closer alignment with expert CDR values, ResUNet++ Medium show that architectural depth, residual pathways, and multi-scale feature aggregation are essential for stable CDR estimation. This is particularly noteworthy in the more challenging high-CDR glaucomatous images, to be found within the DRISHTI and PAPILA datasets The automated CDR values produced by the Medium model is strongly correlated with annotations undertaken by experts, demonstrated moderate predictive value

for glaucoma risk when used alone. When incorporated into a multi-feature classifier alongside structural biomarkers such as cup area, rim area, and vertical/horizontal CDR, predictive performance improved substantially. This factor strengthens the importance of CDR, which is a strong indicator of glaucoma damage but also reinforces the need for more features to be included in regression modelling.

ResUNet++ Medium falls within reported ranges for inter-grader values and therefore supports its suitability as an input into further risk models. In contrast, however the ResUNet++ lighter model exhibited underestimation as well as higher variance, especially with larger cup variation imagery. This supports the need for adequate model complexity to obtain clinically reliable segmentation results.

Overall, the findings validated the primary and secondary hypotheses of superior performance of ResUNet++ for optic cup and disc segmentation and automated CDR correlates strongly with expert grading, and if integrated into multi-feature modelling frameworks would add meaningful contributions to glaucoma risk predictions. The study highlighted that future work should focus attention on high CDR images and integrate this with multi-model imaging.

[Number of words: 7341]

Bibliography

Abraham, N. & Khan, N., 2019. *A Novel Focal Tversky loss function with improved Attention U-Net for lesion segmentation*. s.l., s.n.

Ambtrrn, S. M. P. a. E., 1987. Adaptive Histogram Equalization and its Variations. *Computer Vision Graphics and Image Processing,* 39(3), pp. 355-368.

Bajwa, M. N. et al., 2020. *G1020: A benchmark retinal fundus image dataset for computer-aided glaucoma detection*. s.l., s.n.

Chaurasia, A. K. et al., 2024. Highly Accurate and Precise Automated Cup-to-Disc ratio Quantification for Glaucoma Screening. *Elsevier Opthalmoloy Science,* 4(5).

Chen, Y. B. Y. Z. Y., 2024. Optic disc and cup segmentation for glaucoma detection using Attention U-Net incorporating residual mechanism. *PeerJ Comput Sci,* March.

Hu, J., Shen, L. & Sun, G., 2018. *Squeeze-and-Excitation Networks*. Salt Lake City, s.n., pp. 7132-7141.

Jha, D. a. S. P. H. a. R. M. A. a. J. D. a. d. L. T. a. H. P. a. J., 2019. *ResUNet++: An Advanced Architecture for Medical Image Segmentation*. s.l., s.n., pp. 225 - 2255.

Jha, D. e. a., 2019. *ResUNet++: An advanced architecture for medical image segmentation*. San Diego, s.n.

Jha, D. et al., 2019. *ResUNet++: An advanced architecture for medical image segmentation*.. s.l., s.n., pp. 225-2255.

Kaiming, H., Xiangyu, Z., Ren, S. & .Jian., S., 2015. Deep Residual Learning for Image Recognistion. *Computer Science: Computer Vision and Pattern Recognition*.

Kiefer, R., 2023. *A Catalog of Public Glaucoma Datasets for Machine Learning Applications: A detailed description and analysis of public glaucoma datasets available to machine learning engineers tackling glaucoma-related problems using retinal fundus images and OCT images*.. s.l., s.n.

Kiefer, R., 2023. *A Catalog of Public Glaucoma Datasets for Machine Learning Applications: A detailed description and analysis of public glaucoma datasets available to machine learning engineers tackling glaucoma-related problems using retinal fundus images and OCT images*.. s.l., s.n.

Liu, B. ,. D. S. H., 2021. Joint optic disc and cup segmentation based on densely connected depthwise separable convolution deep network. *National Library of Medicine, National Centre for Biotechnology Information*, January, 21(1).

Manju, K. & Sabeenian, R. S., 2018. Biomedical Research 2018: Robust CDR calculation for glaucoma identification. *Alliedacademies.org*, Issue S137-S144.

Micikevivius, P. & al, e., 2018. Mixed Precision Training. *Computer Science - Artificial Intelligence*.

Morales-Sanchez, O. et al., 2022. PAPILA: Dataset with fundus images and clinical data of both eyes of the same patient for glaucoma assessment.. *Scientific Data*, Volume 291.

OpenXLA, n.d. *Open XLA*. [Online].

Orlando, et al., 2020. REFUGE challenge: A unified framework for evaluating automated methods for glaucoma assessment from fundus photographs.. *Medical Image Analysis*, Volume 101570.

Pal, R., Ghoshal, S. & Chakrabarti, A., 2025. Medical Image Segmentation Using Advanced Attention U-Net. *EasyChair.org*, 10 January.

Pal, R. G. S. a. C. A., 2025. *Medical image segmentation using advanced attention UNet*. Singapore, Springer.

Pizer, S. M. e. a., 1987. Adaptive histogram equalization and its variations. *Computer vision, graphics and image processing,* 39(3), pp. 355-368.

point, A. h., n.d. *Assessment of glaucoma C/D ratio*. s.l., s.n.

Quigley, H. A. B. A. T., n.d. The number of people with glaucoma worldwide in 2010 and 2020. *British Journal of Opthalmology,* 90(3).

Rao, V. e. a., 2025. Segmentation of optic cup and disc regions in optical fundus images for the detection of glaucoma using UNet and a modified loss function. *Machine Learning Computer Science Engineering,* 1(38).

Ronneberger, O., Fischer, P. & Brox, T., 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation.. *Medical Image Computing and Computer-Assisted Intervention (MICCAI),* pp. 234-241.

Sandler, M. et al., 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR),* pp. 4510-4520.

Shalini, R. & Gopi, V. P., 2024. Multiresolution cascaded attention U-Net for localization and segmentation of optic disc and fovea in fundus images. Sci Rep 14, 23107 (2024).. *Sci Rep,* 14(23107).

Sivaswamy, et al., 2014. *Drishti-GS: Retinal image dataset for optic nerve head segmentation*.. s.l., s.n., pp. 53-56.

Sreng, S., Maaneerat, M., Hamamoto, K. & Win, Y. K., 2020. Deep Learning for Optic Disc Segmentation and Glaucoma Diagnosis on Retinal Images. *Applied Sciences,* 10(14).

Tadisetty, et al., 2023. Identifying the Edges of the Optic Cup and the Optic Disc in Glaucoma Patients by Segmentation. *Sensors*, 23(10).

Yadahalli, C., Kumar, C. & Sujatha, S., 2025. Detection Glaucoma using Deep Learning Algorithms. *Procedia Computer Science,* Volume 259, pp. 23-24.

Yu, et al., 2019. Robust optic disc and cup segmentation with deep learning for glaucoma detection. *Comput Med Imaging Graph*.

Zhang, X. et al., 2024. MA-ResUNet: Multi-attention optic cup and optic disc segmentation based on improved U-Net. *IET Image Processing,* 18(12), pp. 3128-3142.

Zhang, X. et al., 2024. MA-ResUNet: Multi segmentation based on improved U-Net.. *IET Image Processing,* 18(12), pp. 456-467.

Zhang, Z. et al., 2010. *ORIGA-light: An online retinal fundus image database for glaucoma analysis and research*.. s.l., s.n., pp. 3065-3068.

Appendix 1: Code

Data Processing Pipeline

```
# Import libraries
import os
import cv2
import numpy as np
```

```
# TensorFlow / Keras
import tensorflow as tf

# Multiprocessing for batch building
from multiprocessing import Pool, cpu_count

# Progress bar
from tqdm import tqdm

# Image processing utilities
from skimage import measure

# For train/val/test split
from sklearn.model_selection import train_test_split

def keep_largest_component(mask, min_area_ratio=0.001):
    """
    Keep only the largest connected component in a binary mask.
    If the largest component is too small relative to the image,
    return an empty mask.

    Parameters:

    mask : np.ndarray
        Binary mask (0/1 or 0/255).
    min_area_ratio : float
        Minimum allowed area of the largest component relative to the image.
        Default 0.001 = 0.1% of the image.

    Returns:

    np.ndarray
        Cleaned binary mask (0 or 1).
    """

    # Ensure binary 0/1
    mask_bin = (mask > 0).astype(np.uint8)

    # No foreground → return empty
    if mask_bin.sum() == 0:
        return np.zeros_like(mask_bin)

    # Connected components
    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask_bin)

    # Only background present
    if num_labels <= 1:
        return mask_bin

    # Largest component (skip background index 0)
    areas = stats[1:, cv2.CC_STAT_AREA]
    largest_label = 1 + np.argmax(areas)
    largest_area = stats[largest_label, cv2.CC_STAT_AREA]

    # Area threshold
    total_area = mask.shape[0] * mask.shape[1]
    if largest_area < min_area_ratio * total_area:
        return np.zeros_like(mask_bin)

    # Return cleaned mask
    cleaned = (labels == largest_label).astype(np.uint8)
    return cleaned


# Define crop ROI
def crop_roi_512(img, od_mask, oc_mask, crop_size=512):
    # Get image dimensions
    h, w = img.shape[:2]

    # Find the optic disc centre
    ys, xs = np.where(od_mask > 0)
    if len(xs) == 0:
        cx, cy = w // 2, h // 2
    else:
        cx = int(xs.mean())
        cy = int(ys.mean())

    # Compute crop boundaries
    half = crop_size // 2
    x1 = max(cx - half, 0)
```

```
    y1 = max(cy - half, 0)
    x2 = min(cx + half, w)
    y2 = min(cy + half, h)

    # Crop the image and masks
    img_crop = img[y1:y2, x1:x2]
    od_crop  = od_mask[y1:y2, x1:x2]
    oc_crop  = oc_mask[y1:y2, x1:x2]

    # Force to 512×512 if edges hit borders
    img_crop = cv2.resize(img_crop, (crop_size, crop_size), interpolation=cv2.INTER_AREA)
    od_crop  = cv2.resize(od_crop,  (crop_size, crop_size),interpolation=cv2.INTER_NEAREST)
    oc_crop  = cv2.resize(oc_crop,  (crop_size, crop_size), nterpolation=cv2.INTER_NEAREST)

    return img_crop, od_crop, oc_crop


# Define worker function
def worker(fundus_path, od_path, oc_path, fname, index):
    return preprocess_512_roi(fundus_path, od_path, oc_path, fname, index)

# Function to append fundus path, OD, OC, filename for each image
args = []
for i, (f, od, oc) in enumerate(zip(fundus_paths, od_paths, oc_paths)):
    fname = os.path.basename(f)
    args.append((f, od, oc, fname, i))


# List and zip fundus_paths, od_paths, oc_paths, filenames
args = list(zip(fundus_paths, od_paths, oc_paths, filenames, range(len(fundus_paths))))
num_workers = max(1, cpu_count() - 1)

# Process each image in parallel to speed up dataset creation
with Pool(processes=num_workers) as pool:
    results = pool.starmap(worker, args)


# Preprocessing for full fundus
def preprocess_512_roi(fundus_path, od_path, oc_path, fname, index,
                       log_file="/content/drive/MyDrive/processed/faint_od_log.txt"):
    # load full-fundus images & raw OD/OC masks
    img = cv2.imread(fundus_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    od_raw = cv2.imread(od_path, cv2.IMREAD_GRAYSCALE)
    oc_raw = cv2.imread(oc_path, cv2.IMREAD_GRAYSCALE)

    # Crop ROI centred on optic disc
    img_roi, od_roi, oc_roi = crop_roi_512(img, od_raw, oc_raw, crop_size=512)

    # Normalize OD/OC masks before thresholding
    od_norm = od_roi.astype(np.float32)
    oc_norm = oc_roi.astype(np.float32)
    od_norm /= od_norm.max() if od_norm.max() > 0 else 1
    oc_norm /= oc_norm.max() if oc_norm.max() > 0 else 1

    # Adaptive thresholding using the 50th percentile
    t_od = np.percentile(od_norm[od_norm > 0], 50) if np.any(od_norm > 0) else 0
    t_oc = np.percentile(oc_norm[oc_norm > 0], 50) if np.any(oc_norm > 0) else 0

    # Keep only the largest connected component
    od_bin = (od_norm >= t_od).astype(np.uint8)
    oc_bin = (oc_norm >= t_oc).astype(np.uint8)

    od_clean = keep_largest_component(od_bin)
    oc_clean = keep_largest_component(oc_bin)

    # Log faint OD masks
    if od_norm.mean() < 0.1:
        with open(log_file, "a") as f:
            f.write(f"Faint OD mask at index {index}, file {fname}\n")

    # Keep images and masks at 512x512
    img_512 = img_roi
    od_512  = od_clean
    oc_512  = oc_clean

    # Apply CLAHE contrast enhancement to the L-channel
    lab = cv2.cvtColor(img_512, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
```

```
    cl = clahe.apply(l)
    img_clahe = cv2.cvtColor(cv2.merge((cl, a, b)), cv2.COLOR_LAB2RGB)

    # Apply Gaussian smoothing and normalize image to [0,1]
    img_gauss = cv2.GaussianBlur(img_clahe, (5,5), 1)
    img_norm = img_gauss.astype(np.float32) / 255.0

    # Stack OD/OC masks
    mask_stack = np.stack([od_512, oc_512], axis=-1).astype(np.uint8)  # then one-hot later

    return img_norm, mask_stack, fname

# Define crop ROI function
def crop_roi_512(img, od_mask, oc_mask, crop_size=512):
# Get image dimensions
h, w = img.shape[:2]
# Find the optic disc centre
    ys, xs = np.where(od_mask > 0)
    if len(xs) == 0:
        cx, cy = w // 2, h // 2
    else:
        cx = int(xs.mean())
        cy = int(ys.mean())

# Compute crop boundaries, centred at cx, cy
    half = crop_size // 2
    x1 = max(cx - half, 0)
    y1 = max(cy - half, 0)
    x2 = min(cx + half, w)
    y2 = min(cy + half, h)

    img_crop = img[y1:y2, x1:x2]
    od_crop  = od_mask[y1:y2, x1:x2]
    oc_crop  = oc_mask[y1:y2, x1:x2]

    # force to 512×512 if edges hit borders
    img_crop = cv2.resize(img_crop, (crop_size, crop_size), interpolation=cv2.INTER_AREA)
    od_crop  = cv2.resize(od_crop,  (crop_size, crop_size), interpolation=cv2.INTER_NEAREST)
    oc_crop  = cv2.resize(oc_crop,  (crop_size, crop_size), interpolation=cv2.INTER_NEAREST)

    return img_crop, od_crop, oc_crop


def preprocess_512_roi(
    fundus_path, od_path, oc_path, fname, index,
    label=None, dataset=None,
    log_file="/content/drive/MyDrive/processed/faint_od_log.txt"
):
    """
    Preprocess a 512×512 ROI fundus image and its OD/OC masks.

    Output mask channel order:
        mask[..., 0] = OD
        mask[..., 1] = OC
    """

    # Convert TF tensors to Python strings
    def decode(x):
        return x.numpy().decode("utf-8") if isinstance(x, tf.Tensor) else x

    fundus_path = decode(fundus_path)
    od_path     = decode(od_path)
    oc_path     = decode(oc_path)
    fname       = decode(fname)

    # Validation path
    for p, label_name in [(fundus_path, "fundus"), (od_path, "OD mask"), (oc_path, "OC mask")]:
        if not os.path.exists(p):
            raise FileNotFoundError(f"[{label_name}] File not found: {p}")

    # Load fundus ROI
    img = cv2.imread(fundus_path)
    if img is None:
        raise ValueError(f"Failed to read fundus image: {fundus_path}")
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Load OD/OC masks
    od_mask = cv2.imread(od_path, cv2.IMREAD_GRAYSCALE)
    oc_mask = cv2.imread(oc_path, cv2.IMREAD_GRAYSCALE)

    if od_mask is None:
```

```
        raise ValueError(f"Failed to read OD mask: {od_path}")
    if oc_mask is None:
        raise ValueError(f"Failed to read OC mask: {oc_path}")

    # Convert grayscale masks to binary
    od_bin = (od_mask > 0).astype(np.float32)
    oc_bin = (oc_mask > 0).astype(np.float32)

    # --- Log faint OD masks ---
    if od_mask.mean() < 5:
        with open(log_file, "a") as f:
            f.write(f"Faint OD mask at index {index}, file {fname}\n")

    # --- Gaussian smoothing ---
    img = cv2.GaussianBlur(img, (5, 5), 1)

    # --- Normalize to [0, 1] ---
    img = img.astype(np.float32) / 255.0

    # --- Stack OD/OC channels ---
    mask_stack = np.stack([od_bin, oc_bin], axis=-1).astype(np.float32)

    # --- Safety checks ---
    assert mask_stack.shape[-1] == 2
    assert mask_stack.min() >= 0 and mask_stack.max() <= 1

    # Return images, mask stack, filename, label and dataset
    return img, mask_stack, fname, label, dataset

    # Load fundus ROI
    img = cv2.imread(fundus_path)
    if img is None:
        raise ValueError(f"Failed to read fundus image: {fundus_path}")
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Load OD/OC masks
    od_mask = cv2.imread(od_path, cv2.IMREAD_GRAYSCALE)
    oc_mask = cv2.imread(oc_path, cv2.IMREAD_GRAYSCALE)

    if od_mask is None:
        raise ValueError(f"Failed to read OD mask: {od_path}")
    if oc_mask is None:
        raise ValueError(f"Failed to read OC mask: {oc_path}")

    # Convert grayscale masks to binary
    od_bin = (od_mask > 0).astype(np.float32)
    oc_bin = (oc_mask > 0).astype(np.float32)

    # Log faint OD masks ---
    if od_mask.mean() < 5:
        with open(log_file, "a") as f:
            f.write(f"Faint OD mask at index {index}, file {fname}\n")

    # --- Gaussian smoothing ---
    img = cv2.GaussianBlur(img, (5, 5), 1)

    # --- Normalize to [0, 1] ---
    img = img.astype(np.float32) / 255.0

    # Channel order:
    #   channel 0 = OD
    #   channel 1 = OC
        mask_stack = np.stack(
        [od_bin, oc_bin],
        axis=-1
    ).astype(np.float32)

    # --- Safety assertions ---
    assert mask_stack.shape[-1] == 2,
    assert mask_stack.min() >= 0 and mask_stack.max() <= 1,

    return img, mask_stack

# Sort and list batch files
batch_dir = "/content/drive/MyDrive/processed_batches/"

batch_files = sorted([
    os.path.join(batch_dir, f)
    for f in os.listdir(batch_dir)
    if f.endswith(".npz")
])
```

```
print("Found", len(batch_files), "batch files")
```

Models

```
# Copy processed batches from Drive to /content/processed_batches/
# This will speed up training.
from google.colab import drive
drive.mount('/content/drive')
import shutil, os, glob, numpy as np
SRC = "/content/drive/MyDrive/processed_batches
DST = "/content/processed_batches"
os.makedirs(DST, exist_ok=True)
for f in sorted(glob.glob(os.path.join(SRC, "*.npz"))):
hutil.copy2(f, os.path.join(DST, os.path.basename(f)))
print(f"Copied {len(glob.glob(DST+'/*.npz'))} files")

#All models use the same dataset split JSON
# ORDER:
#   model 1 — MobileNetV2-UNet(fastest, run first)
#   model 2 — ResUNet++ Lite(medium speed)
#   model 3 — ResUNet++ Medium-Heavy (slowest, run last)

# Dataset pipeline is the same for all models except for batch_size
# MobileNetV2-UNet: batch size 4; ResUNet++ Lite: batch size ; ResUNet++ Medium-Heavy: batch size 1
def npz_generator_s1(file_list):
    for f in file_list:
        data  = np.load(f)
        imgs  = data["images"].astype(np.float32)
        masks = data["masks"].astype(np.float32)
        if imgs.max() > 1.0:
            imgs = imgs / 255.0
        if masks.shape[-1] == 2:
            bg    = np.clip(1.0 - masks[..., 0:1] - masks[..., 1:2], 0.0, 1.0)
            masks = np.concatenate([bg, masks], axis=-1)
        for i in range(imgs.shape[0]):
            yield imgs[i], masks[i]

def augment_fn_s1(image, mask):
    if tf.random.uniform(()) > 0.5:
        image = tf.image.flip_left_right(image); mask = tf.image.flip_left_right(mask)
    if tf.random.uniform(()) > 0.5:
        image = tf.image.flip_up_down(image);    mask = tf.image.flip_up_down(mask)
    image = tf.image.random_brightness(image, 0.1)
    image = tf.image.random_contrast(image, 0.9, 1.1)
    return tf.clip_by_value(image, 0.0, 1.0), mask

def make_dataset_s1(files, augment=False):
    ds = tf.data.Dataset.from_generator(
        lambda: npz_generator_s1(files),
        output_signature=(
            tf.TensorSpec(shape=(512, 512, 3), dtype=tf.float32),
            tf.TensorSpec(shape=(512, 512, 3), dtype=tf.float32),
        ))
    if augment:
        ds = ds.map(augment_fn_s1, num_parallel_calls=tf.data.AUTOTUNE)
    return ds.repeat().batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

def count_samples(files):
    return sum(np.load(p, mmap_mode="r")["images"].shape[0] for p in files)

# Load split
assert os.path.exists(SPLIT_JSON), f"Split JSON not found: {SPLIT_JSON}"
with open(SPLIT_JSON) as f:
    split = json.load(f)

train_files, test_files, y_train, y_test = train_test_split(
    deduped,
    labels,
    test_size=0.15,
    random_state=RANDOM_STATE,
    stratify=labels
)

train_files, val_files, y_train, y_val = train_test_split(
    train_files,
    y_train,
    test_size=0.15,
```

```
    random_state=RANDOM_STATE,
    stratify=y_train
)

train_n          = count_samples(train_files)
val_n            = count_samples(val_files)
steps_per_epoch  = train_n // BATCH_SIZE
validation_steps = val_n   // BATCH_SIZE

print(f"Train: {train_n}  Val: {val_n}  steps/epoch: {steps_per_epoch}")

train_ds_s1 = make_dataset_s1(train_files, augment=True)
val_ds_s1   = make_dataset_s1(val_files,   augment=False)



# Configuration
INPUT_SHAPE = (512, 512, 3)
NUM_CLASSES = 3
LR = 1e-4

# Optimize
tf.keras.backend.clear_session()
gc.collect()
tf.keras.mixed_precision.set_global_policy("mixed_float16")

# Shared layers
class ClipActivation(tf.keras.layers.Layer):
    def __init__(self, min_val=-10.0, max_val=10.0, **kwargs):
        super().__init__(**kwargs)
        self.min_val = min_val
        self.max_val = max_val
    def call(self, x):
        return tf.clip_by_value(x, self.min_val, self.max_val)

def LN():
    return tf.keras.layers.LayerNormalization(epsilon=1e-5)

# Loss functions
def dice_stable(y_true, y_pred, smooth=1e-6):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.clip_by_value(tf.cast(y_pred, tf.float32), smooth, 1.0 - smooth)
    y_true_f = tf.reshape(y_true, [tf.shape(y_true)[0], -1])
    y_pred_f = tf.reshape(y_pred, [tf.shape(y_pred)[0], -1])
    inter = tf.reduce_sum(y_true_f * y_pred_f, axis=1)
    denom = tf.reduce_sum(y_true_f, axis=1) + tf.reduce_sum(y_pred_f, axis=1)
    return tf.reduce_mean((2.0 * inter + smooth) / (denom + smooth))

def dice_disc(y_true, y_pred): return dice_stable(y_true[..., 1], y_pred[..., 1])
def dice_cup(y_true, y_pred):  return dice_stable(y_true[..., 2], y_pred[..., 2])
def dice_macro(y_true, y_pred): return 0.5 * (dice_disc(y_true, y_pred) + dice_cup(y_true, y_pred))

def dice_oc_weighted(y_true, y_pred, smooth=1e-6, oc_weight=2.0):
    y_true = tf.cast(y_true[..., 2], tf.float32)
    y_pred = tf.clip_by_value(tf.cast(y_pred[..., 2], tf.float32), smooth, 1.0 - smooth)
    y_true_f = tf.reshape(y_true, [tf.shape(y_true)[0], -1])
    y_pred_f = tf.reshape(y_pred, [tf.shape(y_pred)[0], -1])
    inter = tf.reduce_sum(y_true_f * y_pred_f, axis=1)
    denom = tf.reduce_sum(y_true_f, axis=1) + tf.reduce_sum(y_pred_f, axis=1)
    return tf.reduce_mean(oc_weight * (2.0 * inter + smooth) / (denom + smooth))

def hybrid_loss(y_true, y_pred,
                alpha=0.7, beta=0.3, gamma=0.75,
                ce_weight=0.5, tv_weight=0.5,
                smooth=1e-6):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.clip_by_value(tf.cast(y_pred, tf.float32), smooth, 1.0 - smooth)

    ce = tf.reduce_mean(
        tf.keras.losses.categorical_crossentropy(y_true, y_pred, from_logits=False)
    )

    num_classes = tf.shape(y_true)[-1]
    yt = tf.reshape(y_true, [-1, num_classes])
    yp = tf.reshape(y_pred, [-1, num_classes])

    tp = tf.reduce_sum(yt * yp, axis=0)
    fp = tf.reduce_sum((1 - yt) * yp, axis=0)
    fn = tf.reduce_sum(yt * (1 - yp), axis=0)
```

```
    tversky = (tp + smooth) / (tp + alpha * fp + beta * fn + smooth)

    t_disc = tversky[1]
    t_cup  = tversky[2]
    t_fg = 0.5 * t_disc + 0.5 * t_cup

    focal_tv = tf.pow(1.0 - t_fg, gamma)
    return ce_weight * ce + tv_weight * focal_tv
```

MobileNetV2-UNet

```
#   MobileNetV2-UNet
def conv_dw_block(x, filters):
    x = tf.keras.layers.SeparableConv2D(filters, 3, padding="same", use_bias=False)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation("relu")(x)

    x = tf.keras.layers.SeparableConv2D(filters, 3, padding="same", use_bias=False)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation("relu")(x)
    return x

def build_mobilenet_unet_v3():
    backbone = tf.keras.applications.MobileNetV2(
        input_shape=INPUT_SHAPE, include_top=False, weights="imagenet"
    )

    skip_names = [
        "block_1_expand_relu",
        "block_3_expand_relu",
        "block_6_expand_relu",
        "block_13_expand_relu",
    ]
    skips = [backbone.get_layer(n).output for n in skip_names]
    bottleneck = backbone.get_layer("block_16_project").output

    x = bottleneck

    x = tf.keras.layers.UpSampling2D()(x)
    x = tf.keras.layers.Concatenate()([x, skips[3]])
    x = conv_dw_block(x, 256)

    x = tf.keras.layers.UpSampling2D()(x)
    x = tf.keras.layers.Concatenate()([x, skips[2]])
    x = conv_dw_block(x, 128)

    x = tf.keras.layers.UpSampling2D()(x)
    x = tf.keras.layers.Concatenate()([x, skips[1]])
    x = conv_dw_block(x, 64)

    x = tf.keras.layers.UpSampling2D()(x)
    x = tf.keras.layers.Concatenate()([x, skips[0]])
    x = conv_dw_block(x, 32)

    x = tf.keras.layers.UpSampling2D()(x)
    x = tf.keras.layers.SeparableConv2D(32, 3, padding="same")(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation("relu")(x)

    outputs = tf.keras.layers.Conv2D(NUM_CLASSES, 1)(x)
    outputs = tf.keras.layers.Activation("softmax", dtype="float32")(outputs)

    return tf.keras.Model(inputs=backbone.input, outputs=outputs, name="MobileNetV2_UNet")
```

ResUNet++ Lite

```
#   ResUNet++ Lite
def res_block_lite(x, filters):
    shortcut = x
    x = tf.keras.layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = LN()(x)
    x = tf.keras.layers.Activation("relu")(x)
    x = ClipActivation()(x)

    x = tf.keras.layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = LN()(x)

    if shortcut.shape[-1] != filters:
        shortcut = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(shortcut)

    x = tf.keras.layers.Add()([x, shortcut])
```

```
    x = tf.keras.layers.Activation("relu")(x)
    x = ClipActivation()(x)
    return x

def se_block_lite(x, r=8):
    filters = x.shape[-1]
    se = tf.keras.layers.GlobalAveragePooling2D()(x)
    se = tf.keras.layers.Dense(max(1, filters // r), activation="relu")(se)
    se = tf.keras.layers.Dense(filters, activation="sigmoid")(se)
    se = tf.keras.layers.Reshape((1, 1, filters))(se)
    return tf.keras.layers.Multiply()([x, se])

def dense_block_lite(x, filters, use_se=True):
    c1 = res_block_lite(x, filters)
    c2 = res_block_lite(tf.keras.layers.Concatenate()([x, c1]), filters)
    out = tf.keras.layers.Concatenate()([x, c1, c2])
    out = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(out)
    out = LN()(out)
    out = tf.keras.layers.Activation("relu")(out)
    out = ClipActivation()(out)
    if use_se:
        out = se_block_lite(out)
    return out

def attention_gate_lite(x, g, filters):
    theta_x = tf.keras.layers.Conv2D(filters, 2, strides=2, padding="same", use_bias=False)(x)
    phi_g   = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(g)
    act     = tf.keras.layers.Add()([theta_x, phi_g])
    act     = tf.keras.layers.ReLU()(act)
    act     = ClipActivation()(act)
    psi     = tf.keras.layers.Conv2D(1, 1, activation="sigmoid",
                                     kernel_initializer="glorot_uniform")(act)
    return tf.keras.layers.Multiply()([x, tf.keras.layers.UpSampling2D()(psi)])

def build_lite_model_v3():
    inputs = tf.keras.Input(INPUT_SHAPE)

    e1 = dense_block_lite(inputs, 32);  p1 = tf.keras.layers.MaxPooling2D()(e1)
    e2 = dense_block_lite(p1,    64);   p2 = tf.keras.layers.MaxPooling2D()(e2)
    e3 = dense_block_lite(p2,   128);   p3 = tf.keras.layers.MaxPooling2D()(e3)
    e4 = dense_block_lite(p3,   256);   p4 = tf.keras.layers.MaxPooling2D()(e4)

    b  = dense_block_lite(p4, 512)

    g4 = attention_gate_lite(e4, b, 256); d4 = tf.keras.layers.UpSampling2D()(b)
    d4 = tf.keras.layers.Concatenate()([d4, g4]); d4 = dense_block_lite(d4, 256)

    g3 = attention_gate_lite(e3, d4, 128); d3 = tf.keras.layers.UpSampling2D()(d4)
    d3 = tf.keras.layers.Concatenate()([d3, g3]); d3 = dense_block_lite(d3, 128)

    g2 = attention_gate_lite(e2, d3,  64); d2 = tf.keras.layers.UpSampling2D()(d3)
    d2 = tf.keras.layers.Concatenate()([d2, g2]); d2 = dense_block_lite(d2,  64)

    g1 = attention_gate_lite(e1, d2,  32); d1 = tf.keras.layers.UpSampling2D()(d2)
    d1 = tf.keras.layers.Concatenate()([d1, g1]); d1 = dense_block_lite(d1,  32)

    outputs = tf.keras.layers.Conv2D(NUM_CLASSES, 1)(d1)
    outputs = tf.keras.layers.Activation("softmax", dtype="float32")(outputs)

    return tf.keras.Model(inputs, outputs, name="ResUNetPP_Lite")
```

ResUNet++

```
#   ResUNet++
def se_block(x, r=8):
    filters = x.shape[-1]
    se = tf.keras.layers.GlobalAveragePooling2D()(x)
    se = tf.keras.layers.Dense(max(1, filters // r), activation="relu")(se)
    se = tf.keras.layers.Dense(filters, activation="sigmoid")(se)
    se = tf.keras.layers.Reshape((1, 1, filters))(se)
    return tf.keras.layers.Multiply()([x, se])

def res_block(x, filters, use_se=True):
    shortcut = x
    x = tf.keras.layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = LN()(x)
    x = tf.keras.layers.Activation("relu")(x)
    x = ClipActivation()(x)

    x = tf.keras.layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
    x = LN()(x)
```

```
    if shortcut.shape[-1] != filters:
        shortcut = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(shortcut)

    x = tf.keras.layers.Add()([x, shortcut])
    x = tf.keras.layers.Activation("relu")(x)
    x = ClipActivation()(x)

    if use_se:
        x = se_block(x)
    return x

def double_res_block(x, filters, use_se=True):
    x = res_block(x, filters, use_se=use_se)
    x = res_block(x, filters, use_se=use_se)
    return x

def aspp_block(x, filters):
    h = INPUT_SHAPE[0] // 16
    w = INPUT_SHAPE[1] // 16

    b0 = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(x)
    b0 = LN()(b0); b0 = tf.keras.layers.Activation("relu")(b0)

    b1 = tf.keras.layers.Conv2D(filters, 3, padding="same", dilation_rate=3, use_bias=False)(x)
    b1 = LN()(b1); b1 = tf.keras.layers.Activation("relu")(b1)

    b2 = tf.keras.layers.Conv2D(filters, 3, padding="same", dilation_rate=6, use_bias=False)(x)
    b2 = LN()(b2); b2 = tf.keras.layers.Activation("relu")(b2)

    b3 = tf.keras.layers.Conv2D(filters, 3, padding="same", dilation_rate=12, use_bias=False)(x)
    b3 = LN()(b3); b3 = tf.keras.layers.Activation("relu")(b3)

    gap = tf.keras.layers.GlobalAveragePooling2D()(x)
    gap = tf.keras.layers.Reshape((1, 1, x.shape[-1]))(gap)
    gap = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(gap)
    gap = LN()(gap); gap = tf.keras.layers.Activation("relu")(gap)
    gap = tf.keras.layers.UpSampling2D(size=(h, w))(gap)

    x = tf.keras.layers.Concatenate()([b0, b1, b2, b3, gap])
    x = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(x)
    x = LN()(x)
    x = tf.keras.layers.Activation("relu")(x)
    x = ClipActivation()(x)
    return x

def attention_gate(x, g, filters):
    theta_x = tf.keras.layers.Conv2D(filters, 2, strides=2, padding="same", use_bias=False)(x)
    phi_g   = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(g)
    act     = tf.keras.layers.Add()([theta_x, phi_g])
    act     = tf.keras.layers.ReLU()(act)
    act     = ClipActivation()(act)
    act     = tf.keras.layers.Conv2D(filters, 1, padding="same", use_bias=False)(act)
    act     = tf.keras.layers.ReLU()(act)
    psi     = tf.keras.layers.Conv2D(1, 1, activation="sigmoid",
                                     kernel_initializer="glorot_uniform")(act)
    return tf.keras.layers.Multiply()([x, tf.keras.layers.UpSampling2D()(psi)])

def build_medium_model_v3():
    inputs = tf.keras.Input(INPUT_SHAPE)

    e1 = double_res_block(inputs,  32); p1 = tf.keras.layers.MaxPooling2D()(e1)
    e2 = double_res_block(p1,      64); p2 = tf.keras.layers.MaxPooling2D()(e2)
    e3 = double_res_block(p2,     128); p3 = tf.keras.layers.MaxPooling2D()(e3)
    e4 = double_res_block(p3,     256); p4 = tf.keras.layers.MaxPooling2D()(e4)

    b  = aspp_block(p4, 512)

    g4 = attention_gate(e4, b, 256); d4 = tf.keras.layers.UpSampling2D()(b)
    d4 = tf.keras.layers.Concatenate()([d4, g4]); d4 = double_res_block(d4, 256)

    g3 = attention_gate(e3, d4, 128); d3 = tf.keras.layers.UpSampling2D()(d4)
    d3 = tf.keras.layers.Concatenate()([d3, g3]); d3 = double_res_block(d3, 128)

    g2 = attention_gate(e2, d3,  64); d2 = tf.keras.layers.UpSampling2D()(d3)
    d2 = tf.keras.layers.Concatenate()([d2, g2]); d2 = double_res_block(d2,  64)

    g1 = attention_gate(e1, d2,  32); d1 = tf.keras.layers.UpSampling2D()(d2)
    d1 = tf.keras.layers.Concatenate()([d1, g1]); d1 = double_res_block(d1,  32)

    outputs = tf.keras.layers.Conv2D(NUM_CLASSES, 1)(d1)
```

```
    outputs = tf.keras.layers.Activation("softmax", dtype="float32")(outputs)

    return tf.keras.Model(inputs, outputs, name="ResUNetPP ")

# Define compile model function
def compile_model(model, lr=1e-4):
    opt = tf.keras.mixed_precision.LossScaleOptimizer(
        tf.keras.optimizers.Adam(lr)
    )

model.compile(
    optimizer=opt,
    loss=hybrid_loss,
       metrics=[dice_disc, dice_cup, dice_macro, dice_oc_weighted]
)
return model

# Build and compile medium model
model_medium = build_medium_model()
compile_model(model_medium, lr=1e-4)
model_medium.summary()

# Define callbacks function
def build_callbacks(save_path, log_name):
    return [
        tf.keras.callbacks.EarlyStopping(
            monitor="val_dice_macro",
            patience=10,
            mode="max",
            restore_best_weights=True
        ),
        tf.keras.callbacks.ModelCheckpoint(
            save_path,
            monitor="val_dice_macro",
            save_best_only=True,
            mode="max",
            verbose=1
        ),
        tf.keras.callbacks.ReduceLROnPlateau(
            monitor="val_dice_macro",
            factor=0.5,
            patience=5,
            min_lr=1e-7,
            mode="max",
            verbose=1
        ),
        tf.keras.callbacks.CSVLogger(
            os.path.join(CHECKPOINT_DIR, log_name),
            append=True
        ),
    ]

# Define train model function
def train_model(model, train_ds, val_ds, steps_per_epoch, validation_steps,
                save_path, log_name, epochs=100):

    callbacks = build_callbacks(save_path, log_name)

    model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs,
        steps_per_epoch=steps_per_epoch,
        validation_steps=validation_steps,
        callbacks=callbacks,
        verbose=1
    )

# Train mobile model
train_model(
    model_mobile,
    train_ds_s1, val_ds_s1,
    steps_per_epoch, validation_steps,
    os.path.join(CHECKPOINT_DIR, "best_mobilenetv2_unet.keras"),
    "log_mobilenet.csv"
)
```

```
# Train lite model
train_model(
    model_lite,
    train_ds_s2, val_ds_s2,
    steps_per_epoch, validation_steps,
    os.path.join(CHECKPOINT_DIR, "best_resunetpp_lite.keras"),
    "log_lite.csv"
)

# Train medium model
train_model(
    model_medium,
    train_ds_s3, val_ds_s3,
    steps_per_epoch, validation_steps,
    os.path.join(CHECKPOINT_DIR, "best_resunetpp_medium.keras"),
    "log_medium.csv"
)
```